

บทที่ 9

รีเกรสชัน

- ✓ 9.1 แนะนำรีเกรสชัน
- ✓ 9.2 K-Nearest Neighbours (KNN)
- ✓ 9.3 Ordinary Least Squares (OLS)
- ✓ 9.4 Decision Tree (DT)
- ✓ 9.5 Artificial Neural Network (ANN)
- ✓ 9.6 Support Vector Regression (SVR)
- ✓ 9.7 แบบฝึกหัด

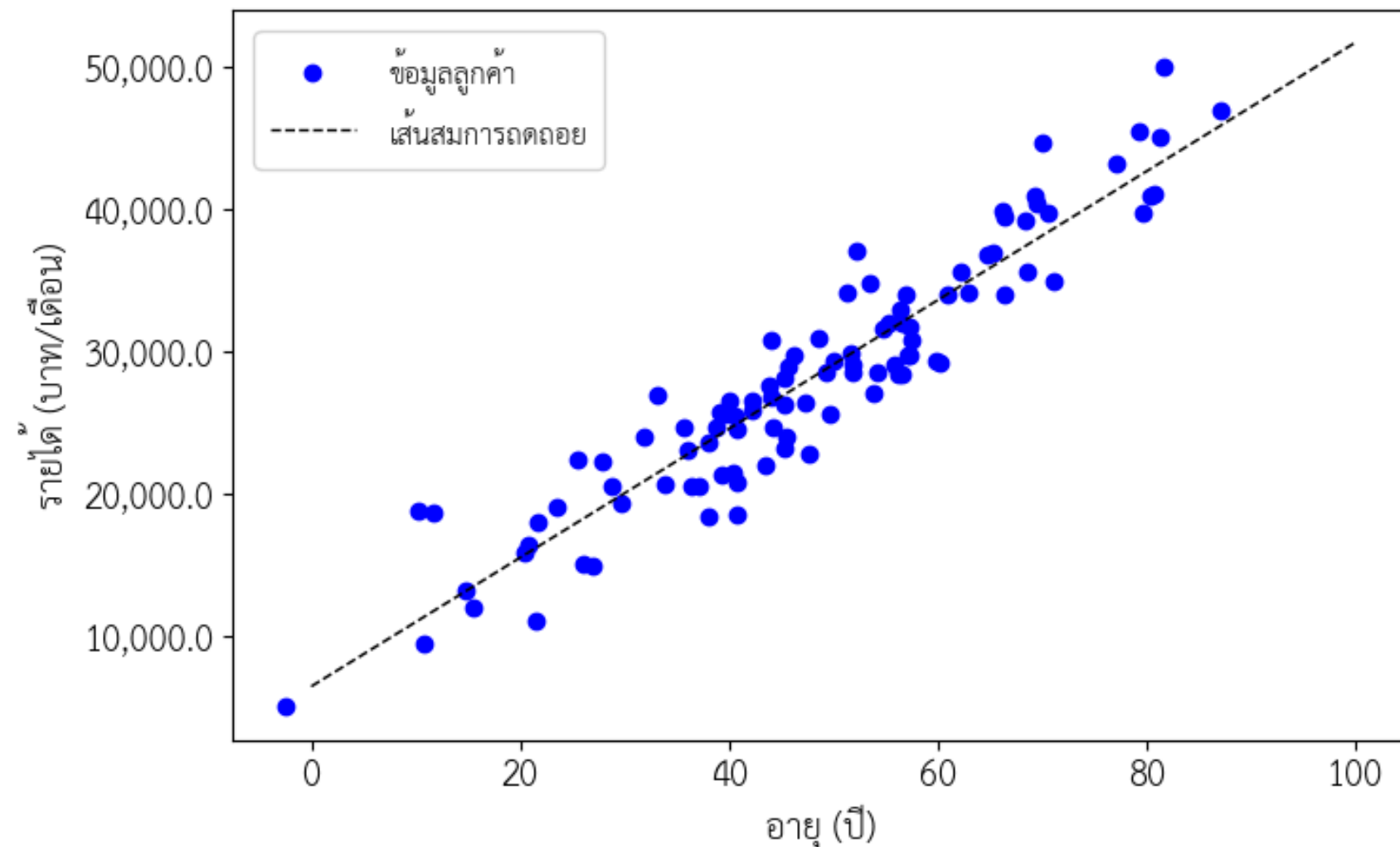


9.1 แนะนำรีเกรสชัน

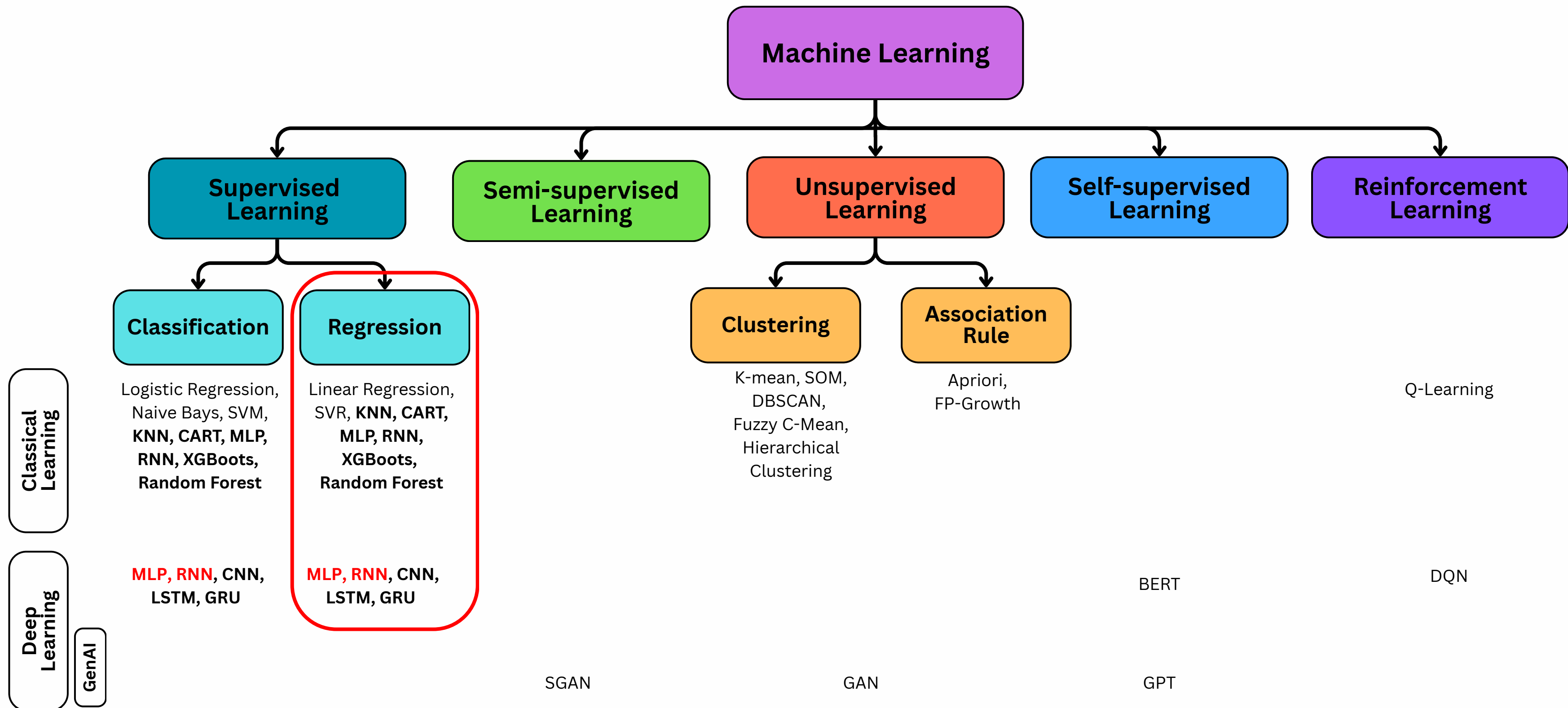
- **รีเกรสชัน (Regression)** เป็นเทคนิคการทำนายค่าตัวเลขต่อเนื่อง (Continuous Value) โดยเรียนรู้ความสัมพันธ์ระหว่างตัวแปรต้น (Independent/Input) และตัวแปรตาม (Dependent/Output) จากข้อมูลที่มีอยู่
- เป็นเทคนิคหนึ่งของ ML ในกลุ่ม Supervised Learning
- ตัวแปรตาม (Dependent/Target) มีลักษณะเป็นตัวแปรเชิงปริมาณ (Quantitative/Numerical)

- **ตัวอย่างการประยุกต์ใช้**

- การประเมินราคาอสังหาริมทรัพย์ (บาท/ตารางเมตร)
- การประมาณการรายได้ (บาท/เดือน)
- การทำนายราคาหุ้น (บาท)
- การทำนายอุณหภูมิ (องศาเซลเซียส)
- การทำนายปริมาณน้ำฝน (มม./24 ชม.)
- การทำนายปริมาณฝุ่นละออง PM 2.5 (มคก./ลบ.ม.)



ภาพที่ 9-1 : ตัวอย่างการทำนายค่าตัวเลขต่อเนื่อง



ภาพที่ 9-2 : Machine Learning Techniques

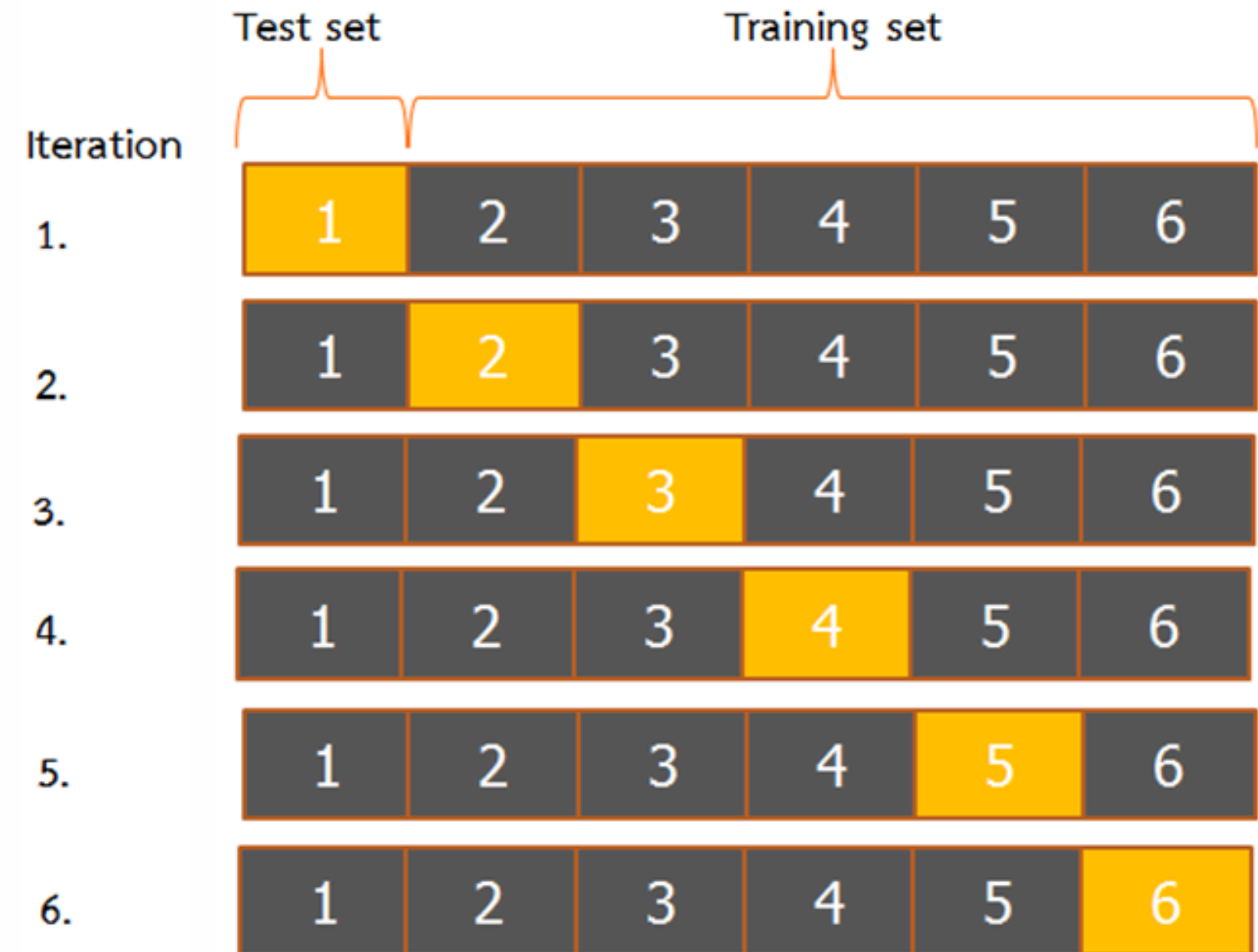
9.1.1 Model Validation

การทดสอบ (Validation) เป็นวิธีการที่ช่วยให้การประเมินประสิทธิภาพ (Efficiency) และประสิทธิผล (Effectiveness) ของโมเดล ที่สร้างขึ้นมีความน่าเชื่อถือมากขึ้น ประกอบด้วยเทคนิคต่าง ๆ ดังนี้

- Hold Out
- Repeat Random Sub-sampling
- Cross-Validation

ขั้นตอนการแบ่งข้อมูลด้วย k -fold Cross Validation

- สุ่มแยกข้อมูลจาก dataset ออกเป็น k กอง (ในที่นี้กำหนด $k=6$)
- รอบแรกใช้กองที่ 1 เป็นข้อมูลสำหรับ Test set แล้วนำ 5 กองที่เหลือมารวมกันเพื่อใช้เป็น Training test
- รอบสองใช้กองที่ 2 เป็น test set แล้วใช้ 5 กองที่เหลือรวมเป็น Training set
- ทำเช่นนี้ไปจนครบ 6 รอบ
- ในแต่ละรอบจะใช้ข้อมูล Test set เพื่อวัดประสิทธิภาพของโมเดล ด้วย Evaluation Metrics เช่น MAPE, Accuracy, Precision, Recall แล้วหาค่าเฉลี่ยจากทั้ง 6 รอบ เพื่อกำหนดเป็นประสิทธิภาพของโมเดล



ภาพที่ 9-3 : 6-fold cross validation

โค้ดไพทอน สำหรับ 3-fold Cross Validation

```
1  import numpy as np
2  from sklearn.model_selection import KFold
3  x = np.array(
4      [[1, 2],
5       [8, 4],
6       [9, 6],
7       [7, 8],
8       [4, 3]])
9  y = np.array([1, 7, 8, 4, 2])
10 kf = KFold(n_splits=3, shuffle=True, random_state=0).split(x)
11 for i, (train_index, test_index) in enumerate(kf):
12     print(f"Fold {i+1}:")
13     print(f"  Train index={train_index}")
14     print(f"  Test index={test_index}")
15     x_train, x_test = x[train_index], x[test_index]
16     y_train, y_test = y[train_index], y[test_index]
```

Method: KFold

Parameter:

n_splits=3 คือ จำนวนกองของข้อมูลที่ต้องการแบ่ง

shuffle=True คือ กำหนดให้มีการสุ่มข้อมูล

random_state=0 คือ เป็นการสุ่มลำดับเดิมเสมอ

x คือ ข้อมูลตัวแปรต้นทุก records

Output:

train_index คือ Index ของ Training data

test_index คือ Index ของ Test data

Fold 1:

Train index=[1 3 4]

Test index=[0 2]

Fold 2:

Train index=[0 2 4]

Test index=[1 3]

Fold 3:

Train index=[0 1 2 3]

Test index=[4]

9.1.2 Performance Evaluation for Regression

- **Evaluation Metrics**

- Mean Absolute Error (MAE)
- Mean Absolute Percentage Error (MAPE)
- Mean Square Error (MSE)
- Root Mean Square Error (RMSE)
- R-squared (R^2)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$$

$$\text{MAPE} = \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{y_i} \times 100$$

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

เมื่อ

$\hat{y}_i$ คือ ค่าที่ทำนายได้ ของข้อมูลแถว (record) ที่ i

y_i คือ ค่าที่เกิดขึ้นจริง ของข้อมูลแถว (record) ที่ i

n คือ จำนวนข้อมูลทดสอบ (Test Points) ทั้งหมด

- ตัวอย่างการคำนวณ สำหรับ Performance Evaluation for Regression

จงคำนวณค่า MAE, MAPE, MSE, RMSE, R^2 เมื่อค่าที่เกิดขึ้นจริง (y_{test}) และค่าที่ทำนายได้ (y_{pred}) เป็นดังนี้

$y_{\text{test}} = [1, 7, 8, 4, 2]$ #ค่าที่เกิดขึ้นจริง

$y_{\text{pred}} = [2, 6, 10, 3, 2]$ #ค่าที่ทำนายได้

วิธีทำ

$$\begin{aligned}\text{MAE} &= \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i| \\ &= \frac{1}{5} (|2 - 1| + |6 - 7| + |10 - 8| + |3 - 4| + |2 - 2|) = \frac{1}{5} (5) = 1\end{aligned}$$

$$\begin{aligned}\text{MAPE} &= \frac{1}{n} \sum_{i=1}^n \frac{|\hat{y}_i - y_i|}{y_i} \times 100 \\ &= \frac{1}{5} \left(\frac{|2 - 1|}{1} + \frac{|6 - 7|}{7} + \frac{|10 - 8|}{8} + \frac{|3 - 4|}{4} + \frac{|2 - 2|}{2} \right) \times 100 \\ &= \frac{1}{5} (1.6429) \times 100 = 32.86\%\end{aligned}$$

$$\begin{aligned}\text{MSE} &= \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 \\ &= \frac{1}{5} ((2 - 1)^2 + (6 - 7)^2 + (10 - 8)^2 + (3 - 4)^2 + (2 - 2)^2) = \frac{1}{5} (7) = 1.4\end{aligned}$$

$$\begin{aligned}\text{RMSE} &= \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2} \\ &= \sqrt{1.4} = 1.18\end{aligned}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

y_test = [1, 7, 8, 4, 2] #ค่าที่เกิดขึ้นจริง
y_pred = [2, 6, 10, 3, 2] #ค่าที่ทำนายได้

- คำนวณแต่ละส่วนของ R^2

$$\sum_{i=1}^n (\hat{y}_i - y_i)^2 = (2 - 1)^2 + (6 - 7)^2 + (10 - 8)^2 + (3 - 4)^2 + (2 - 2)^2 = 7$$

$$\bar{y} = \frac{1}{5}(1 + 7 + 8 + 4 + 2) = 4.4$$

$$\sum_{i=1}^n (y_i - \bar{y})^2 = (1 - 4.4)^2 + (7 - 4.4)^2 + (8 - 4.4)^2 + (4 - 4.4)^2 + (2 - 4.4)^2 = 37.2$$

- คำนวณรวม

$$R^2 = 1 - \frac{7}{37.2} = 0.81$$

โค้ดไพทอน สำหรับ Performance Evaluation for Regression

```
1 from sklearn.metrics import mean_absolute_error
2 from sklearn.metrics import mean_absolute_percentage_error
3 from sklearn.metrics import mean_squared_error
4 from sklearn.metrics import root_mean_squared_error
5 from sklearn.metrics import r2_score
6
7 y_test=[1, 7, 8, 4, 2] #ค่าที่เกิดขึ้นจริง
8 y_pred=[2, 6, 10, 3, 2] #ค่าที่ทำนายได้จากโมเดล
9
10 mae=mean_absolute_error(y_test , y_pred)
11 mape=mean_absolute_percentage_error(y_test, y_pred)*100
12 mse=mean_squared_error(y_test, y_pred)
13 rmse=root_mean_squared_error(y_test, y_pred)
14 r2 = r2_score(y_test, y_pred)
15
16 print('MAE :', round(mae,2))
17 print('MAPE :', round(mape,2))
18 print('MSE :', round(mse,2))
19 print('RMSE :', round(rmse,2))
20 print('R-squared :', round(r2,2))
```

MAE : 1.0

MAPE : 32.86

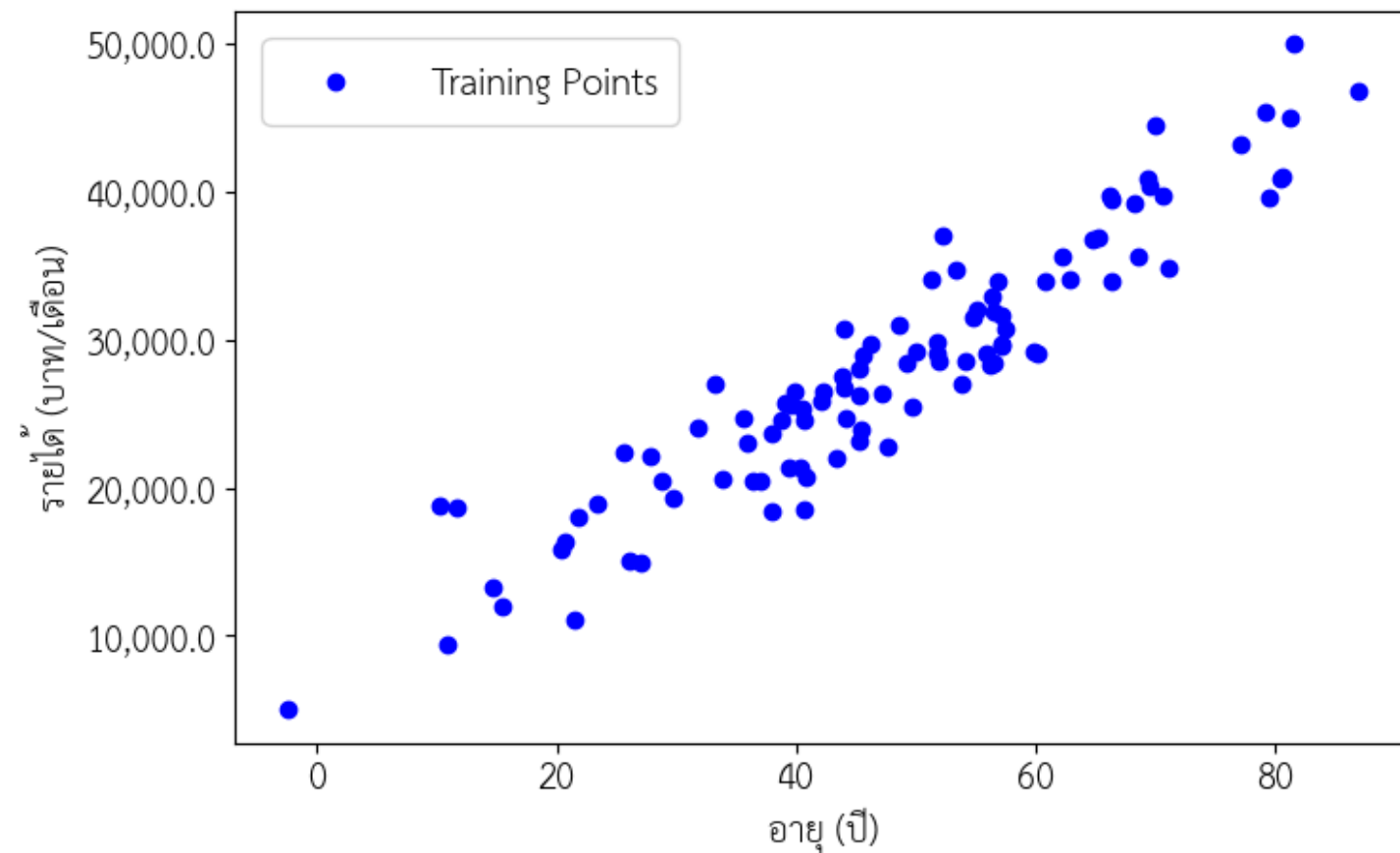
MSE : 1.4

RMSE : 1.18

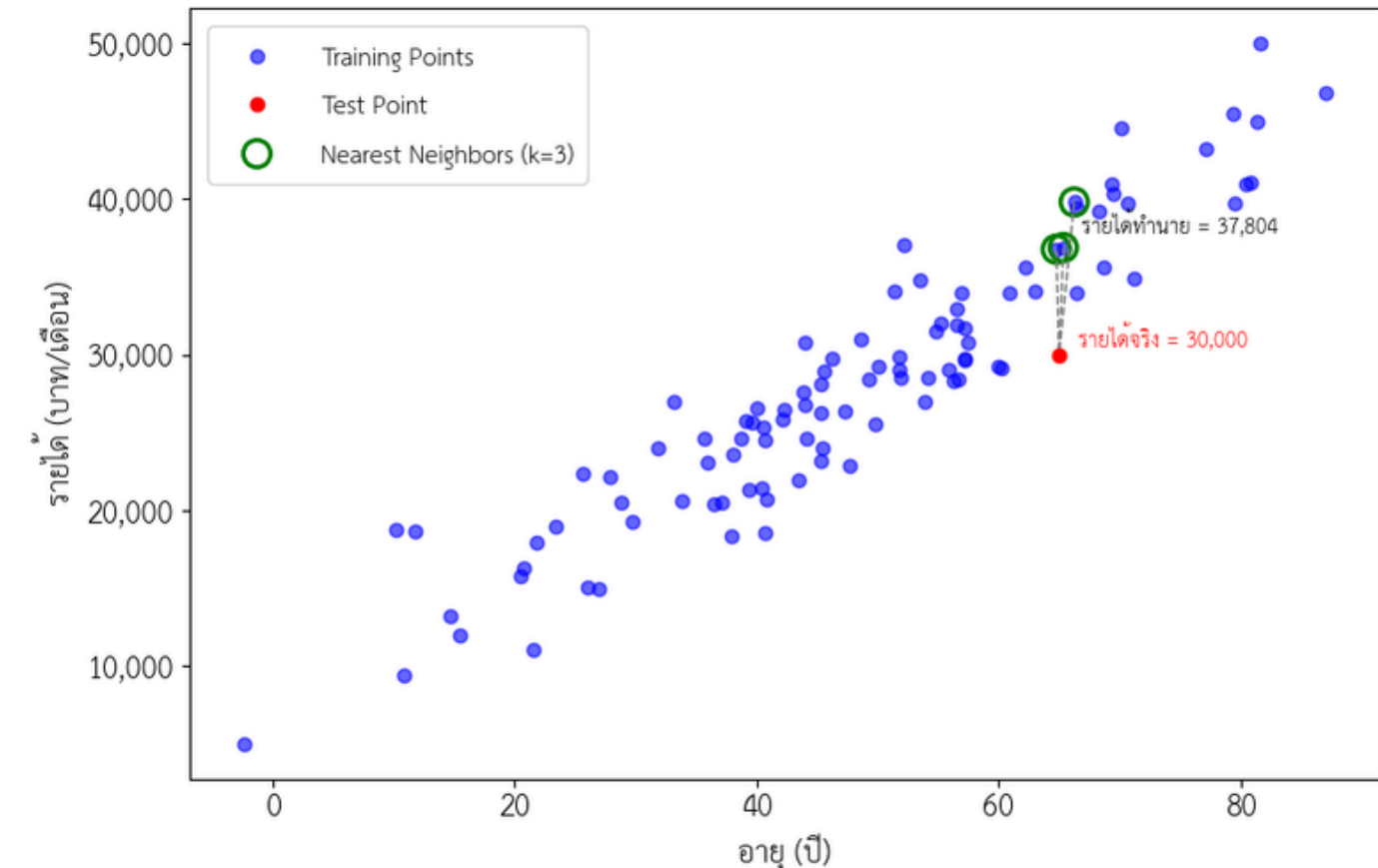
R-squared : 0.81

9.2 K-Nearest Neighbours (KNN)

- อาศัยหลักการวัดระยะทาง ซึ่งนิยมวัดระยะทางโดยใช้วิธี Euclidian Distance
- ง่ายต่อการทำความเข้าใจและการคำนวณ
- Sensitive ต่อ Outliers ทำให้ผลการทำนายผิดพลาด ควสอบ outliers ก่อนหรือทำนายโดย weight ตามระยะทาง
- ข้อมูลที่มีช่วงต่างกันมาก เช่น อายุ [0-100] กับเงินเดือน [1 พัน, 1 แสน] จะทำให้ข้อมูลที่มีช่วงกว้างกว่า มีอิทธิพลต่อการคำนวณระยะทางมากกว่า ควรทำการ Normalize/Standardize ข้อมูล เช่น ทำให้ขอยู่ในช่วง [0, 1]
- หากข้อมูลมีขนาดใหญ่ จะทำให้การทำนายใช้เวลานานมากเพราะต้องคำนวณทุก Training Points



ภาพที่ 9-4ก : ชุดข้อมูลฝึก (Training Set)



ภาพที่ 9-4ข : การทดสอบ (Test)

ภาพที่ 9-4 : ตัวอย่างการประมาณการรายได้ด้วย KNN

9.2.1 การคำนวณสำหรับ KNN

สูตรการคำนวณ

ขั้นที่ 1 : คำนวณระยะทาง (ความแตกต่าง) ระหว่าง test point x กับ ทั้งหมดของ training point x^n

$$d^n = d(x, x^n), \quad n = 1, \dots, N$$

เมื่อ $d(x, x^n)$ **คือ** ระยะทางระหว่าง test point x กับ training point x^n

N **คือ** จำนวน training point ทั้งหมด

หากใช้วิธีวัดระยะทางแบบยุคลิด (Euclidean Distance) จะได้

$$d(x, x^n) = \sqrt{\sum_{i \in m} (x_i - x_i^n)^2}$$

เมื่อ m **คือ** จำนวนตัวแปรต้น, จำนวนปัจจัย (Factor), หรือจำนวนคุณลักษณะ (Feature)

ขั้นที่ 2 : หา training point x^n ซึ่งมีระยะทาง $d(x, x^n)$ ใกล้เคียงกับ test point x มากที่สุด จำนวน k อันดับแรก

ขั้นที่ 3 : กำหนดผลลัพธ์ (Output) สำหรับงาน Regression

$$\hat{y} = \frac{1}{k} \sum_{i \in N_k(x)} y_i$$

เมื่อ $\hat{y}$ **คือ** ค่าที่ทำนายได้ y_i **คือ** ค่าที่เกิดขึ้นจริงของเพื่อนบ้านตัวที่ i $N_k(x)$ **คือ** เซตของเพื่อนบ้าน k อันดับแรก

9.2.2 ตัวอย่างการคำนวณ KNN

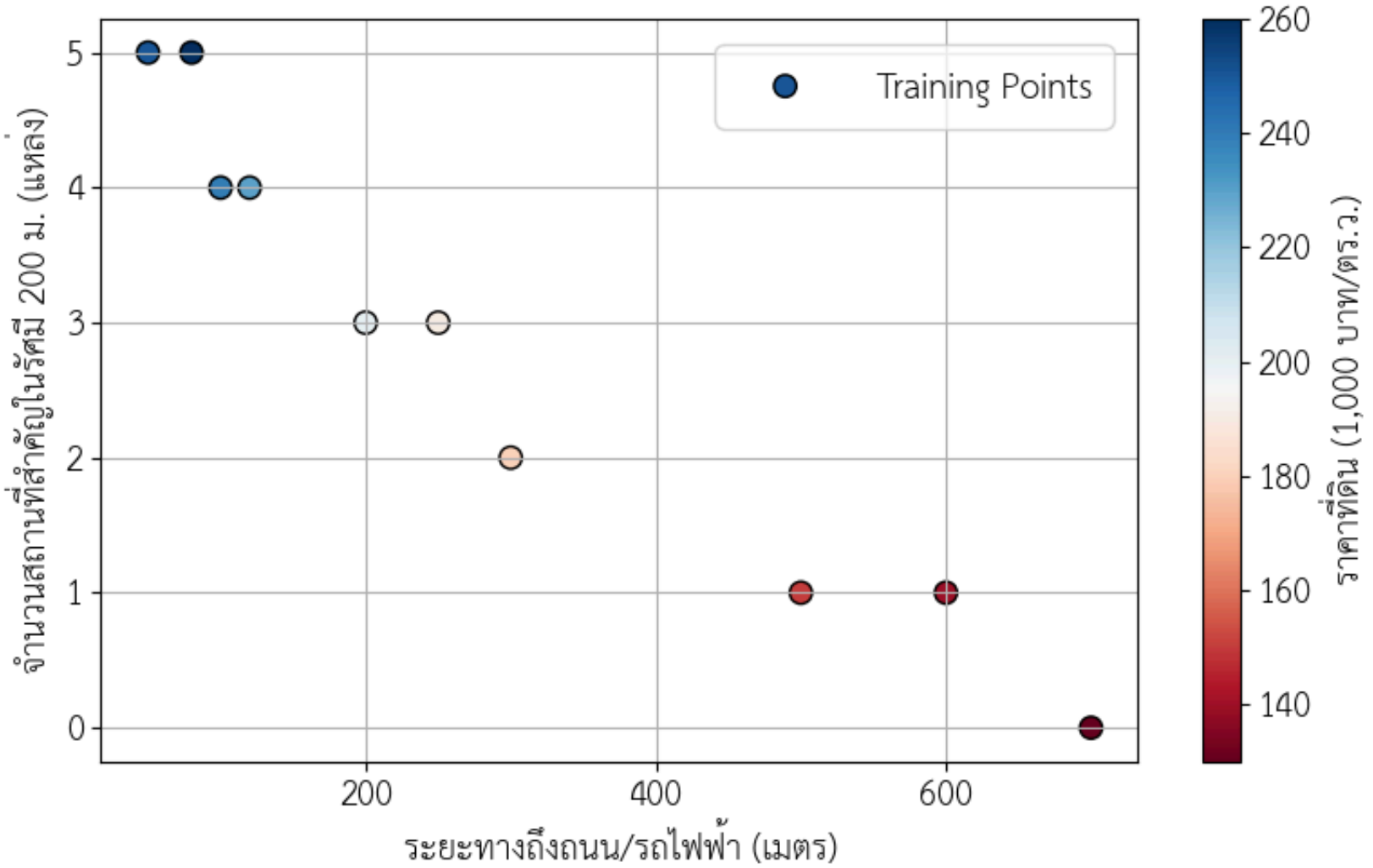
จงแสดงวิธีการคำนวณ KNN เพื่อประเมินราคาที่ดิน

ตารางที่ 9-1ก : ชุดข้อมูลฝึก (Training Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	50	5	250
2	100	4	240
3	80	5	260
4	200	3	200
5	500	1	150
6	300	2	180
7	700	0	130
8	600	1	140
9	250	3	190
10	120	4	230

ตารางที่ 9-1ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	220	2	?



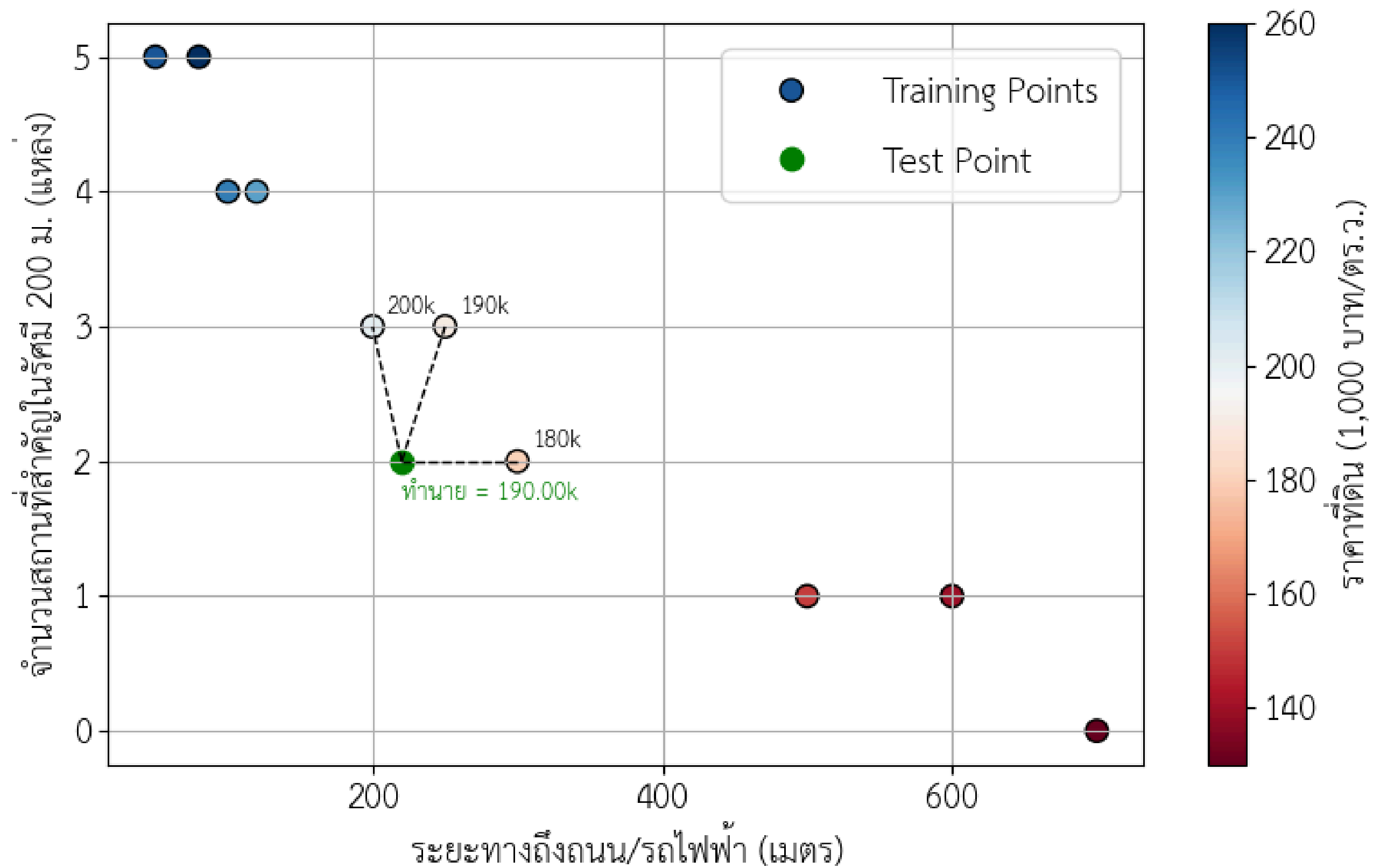
ภาพที่ 9-5 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ชุดข้อมูลทดสอบ (Test Set)						
	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)			
	220	2	190.....	(ค่าเฉลี่ยของคำตอบจากข้อมูลฝึก 3 อันดับแรก)		
ชุดข้อมูลฝึก (Traning Set)						
รายการที่	ปัจจัยที่ 1 (x1)	ปัจจัยที่ 2 (x2)	ผลลัพธ์ (y)	การคำนวณระยะทาง ด้วยวิธีแบบยุคลิด	ระยะทาง	รายการข้อมูลฝึก ที่ใกล้ที่สุด 3 อันดับ
1	50	5	250	$\text{SQRT}((220-50)^2 + (2-5)^2)$	170.03	
2	100	4	240	$\text{SQRT}((220-100)^2 + (2-4)^2)$	120.02	
3	80	5	260	$\text{SQRT}((220-80)^2 + (2-5)^2)$	140.03	
4	200	3	200	$\text{SQRT}((220-200)^2 + (2-3)^2)$	20.02	x
5	500	1	150	$\text{SQRT}((220-500)^2 + (2-1)^2)$	280	
6	300	2	180	$\text{SQRT}((220-300)^2 + (2-2)^2)$	80	x
7	700	0	130	$\text{SQRT}((220-700)^2 + (2-0)^2)$	480	
8	600	1	140	$\text{SQRT}((220-600)^2 + (2-1)^2)$	380	
9	250	3	190	$\text{SQRT}((220-250)^2 + (2-3)^2)$	30.02	x
10	120	4	230	$\text{SQRT}((220-120)^2 + (2-4)^2)$	100.02	

นำผลลัพธ์ 190 ไปคูณ 1,000 เพื่อปรับให้อยู่ในหน่วย บาท/ตร.ว. จะได้ $190 \times 1,000 = 190,000$

ตอบ ราคาที่ดินที่ประเมินได้ คือ 190,000 บาท/ตร.ว.



ภาพที่ 9-6 : การทดสอบ (Test) หรือการประเมินราคาที่ดิน

9.2.3 ตัวอย่างโค้ด KNN

- เขียนโค้ดแบบไม่ใช้ไลบรารี ML

```
1 #Import Python library
2 import math
3 import numpy as np
4
5 #กำหนดค่าเริ่มต้น
6 x_train = [[50, 5],[100, 4], [80, 5], [200, 3], [500, 1], [300, 2], [700, 0], [600, 1], [250, 3],[120, 4]]
7 y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
8 x_test = [220, 2]
9 k = 3
10
11
12 #ขั้นที่ 1 : คำนวณระยะทางระหว่าง test point กับ training points
13 dist = []
14 for i in range(len(x_train)):
15     d = math.sqrt( pow(x_test[0] - x_train[i][0], 2) +
16                   pow(x_test[1] - x_train[i][1], 2) )
17     dist.append(round(d,2))
18
19 print('ขั้นที่ 1 : ระยะทาง คือ', dist)
20
```

```

21 #ขั้นที่ 2 : หา training points ซึ่งมีระยะทางใกล้เคียงกับ test point มากที่สุด จำนวน k อันดับแรก
22 sort_dist = sorted(dist.copy())[:k]
23
24 index = []
25 for s in sort_dist:
26     for i, value in enumerate(dist):
27         if s == value:
28             index.append(i)
29
30     if len(index) == k:
31         break;
32
33 print('ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ', [i + 1 for i in index])
34
35 #ขั้นที่ 3 : หาคำตอบจากตัวแปรตามของ training points ที่ใกล้ที่สุด k ตัว
36 #หาค่า average สำหรับงาน regression
37 y_pred = [y_train[i] for i in index]
38 averaeg = np.mean(y_pred)*1000
39
40 print(f'ขั้นที่ 3   ราคาประเมินที่ดิน คือ {averaeg:,.2f} บาท/ตร.ว.')

```

ขั้นที่ 1 : ระยะทาง คือ [170.03, 120.02, 140.03, 20.02, 280.0, 80.0, 480.0, 380.0, 30.02, 100.02]

ขั้นที่ 2 : รายการข้อมูลฝึก 3 อันดับแรก คือ [4, 9, 6]

ขั้นที่ 3 : ราคาประเมินที่ดิน คือ 190,000.00 บาท/ตร.ว.

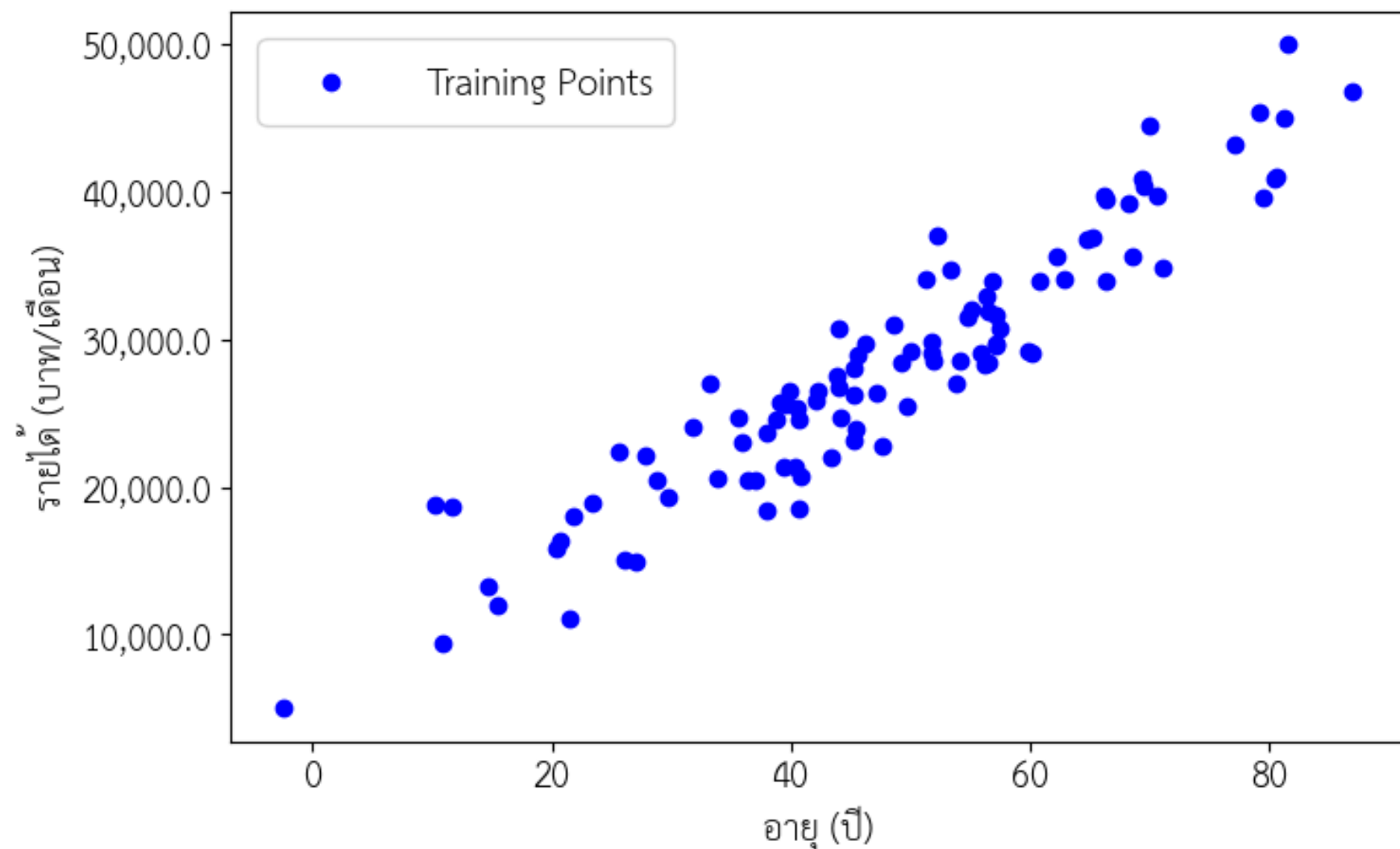
- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python library
2 from sklearn.neighbors import KNeighborsRegressor
3
4 #Load Data
5 x_train = [[50, 5], [100, 4], [80, 5], [200, 3], [500, 1], [300, 2], [700, 0], [600, 1], [250, 3], [120, 4]]
6 y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
7 x_test = [220, 2]
8 k = 3
9
10 #Build Model
11 net = KNeighborsRegressor(n_neighbors = k, metric = 'minkowski', p = 2)
12 net.fit(x_train, y_train)
13
14 #Test Model
15 y_pred = net.predict([x_test])*1000
16 print(f'ราคาประเมินที่ดิน คือ {averaeg:,.2f} บาท/ตร.ว.')
```

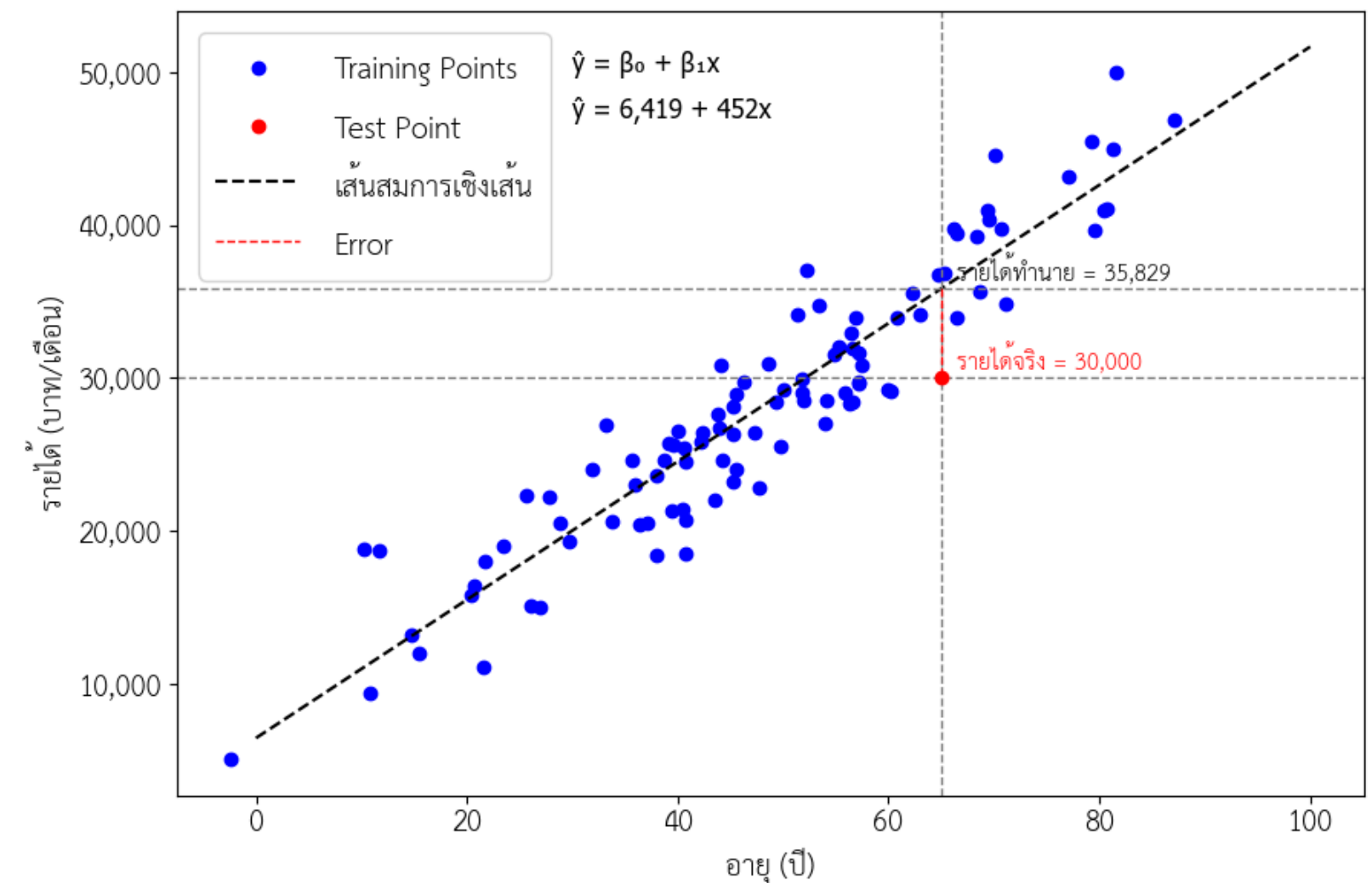
ราคาประเมินที่ดิน คือ 190,000.00 บาท/ตร.ว.

9.3 Ordinary Least Squares (OLS)

- อาศัยหลักการของ Pseudo Inverse ของ Matrix เพื่อประมาณค่าพารามิเตอร์ β ของสมการเชิงเส้น
- เพื่อหาค่าความสัมพันธ์ระหว่างตัวแปรต้น (X) กับ ตัวแปรตาม (Y) โดยให้เส้นตรง มีระยะห่างจากข้อมูลจริงน้อยที่สุด
- ข่ายต่อการทำความเข้าใจและการคำนวณ และเหมาะกับข้อมูลที่มีความสัมพันธ์เชิงเส้น
- Sensitive ต่อ Outliers ส่งผลให้ ค่า β เบี่ยงเบนจากค่าที่ควรจะเป็น ควรลบ outliers หรือ take log



ภาพที่ 9-7ก : ชุดข้อมูลฝึก (Training Set)



ภาพที่ 9-7ข : การทดสอบ (Test)

ภาพที่ 9-7 : ตัวอย่างการประมาณการรายได้ด้วย OLS

9.3.1 การคำนวณสำหรับ OLS

ขั้นที่ 1 : คำนวณซูโดอินเวิร์ส (Pseudo Inverse) ของ $\mathbf{X}$

$$\mathbf{X}^+ = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$$

เมื่อ $\mathbf{X}$ คือ เมตริกของชุดข้อมูลฝึก (ตัวแปรต้น), $\mathbf{X}^T$ คือ ทรานสโพส (Transpose) ของ $\mathbf{X}$

$\mathbf{X}^+$ คือ ซูโดอินเวิร์ส (Pseudo inverse) ของ $\mathbf{X}$

$(\mathbf{X}^T \mathbf{X})^{-1}$ คือ อินเวิร์ส ของผลคูณระหว่าง $\mathbf{X}^T$ กับ $\mathbf{X}$

ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ (Coefficient) ของ $\mathbf{X}$

$$\boldsymbol{\beta} = \mathbf{X}^+ \mathbf{y}$$

$$\boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

เมื่อ $\boldsymbol{\beta}$ คือ เวกเตอร์ของค่าสัมประสิทธิ์

$\mathbf{y}$ คือ เวกเตอร์ของข้อมูลฝึก (ตัวแปรตาม)

ขั้นที่ 3 : นำ $\boldsymbol{\beta}$ มาสร้างเป็นสมการ ได้ดังนี้

$$\hat{y} = \mathbf{x} \boldsymbol{\beta}$$

เมื่อ $\hat{y}$ คือ ผลการทำนาย, $\mathbf{x}$ คือ เวกเตอร์ของข้อมูลทดสอบ และเมื่อกระจายผลคูณจะได้

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_m x_m$$

เมื่อ m คือ จำนวนตัวแปรต้น, β_m คือค่าสัมประสิทธิ์ตัวที่ m , x_m คือค่าของตัวแปรต้นตัวที่ m

เพื่อให้สมการมี β_0 จำเป็นต้องเพิ่มเวกเตอร์ 1 เข้าไปยังเมตริก $\mathbf{X}$ ในขั้นที่ 1

9.3.2 ตัวอย่างการสร้าง OLS

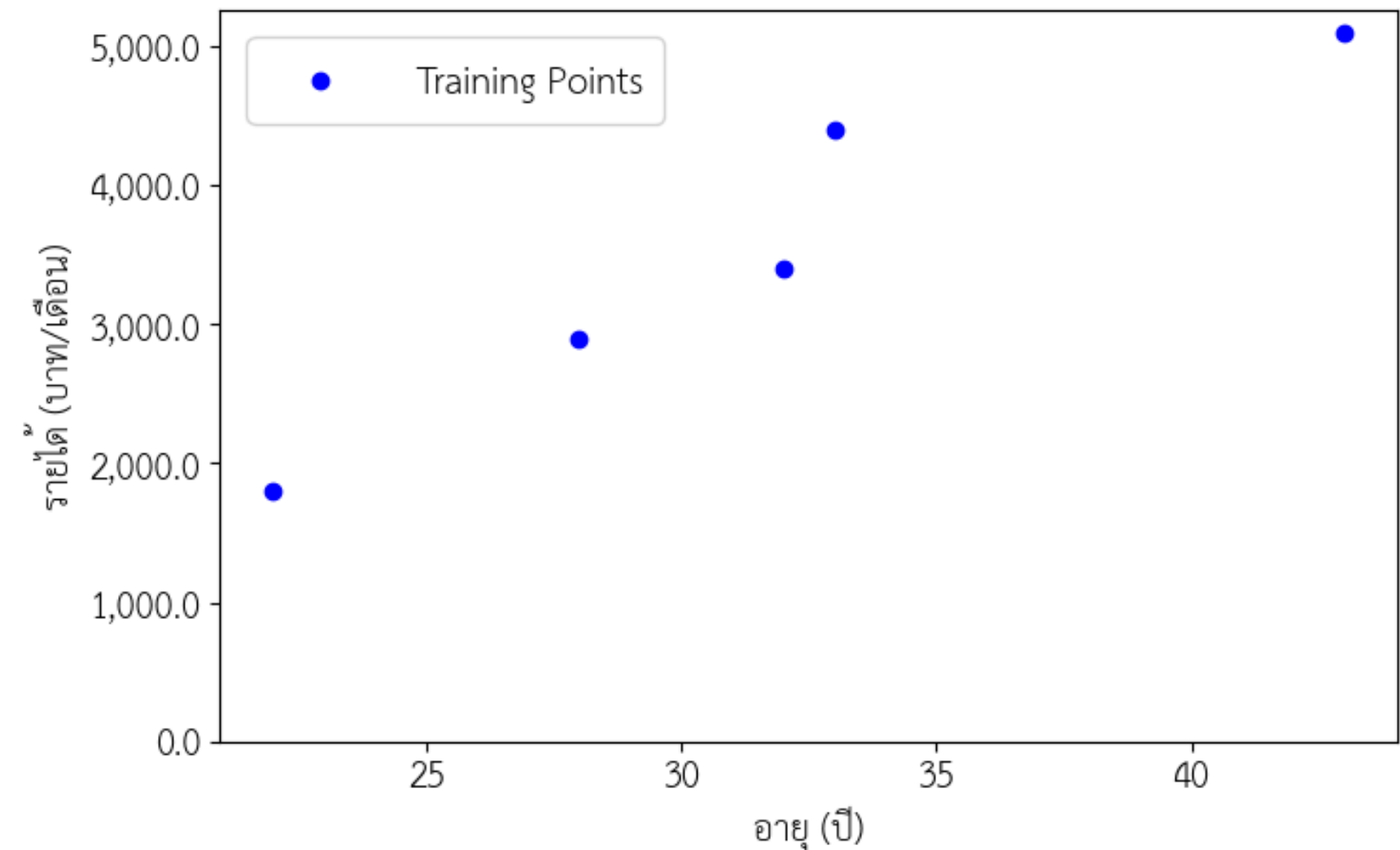
จงแสดงวิธีการคำนวณ OLS เพื่อประมาณการรายได้

ตารางที่ 9-2ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	รายได้ (บาท)
1	22	1800
2	28	2900
3	32	3400
4	33	4400
5	43	5100

ตารางที่ 9-2ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	รายได้ (บาท)
1	20	?
2	30	?



ภาพที่ 9-8 : ชุดข้อมูลฝึก (Training Set)

วิธีทำ

ขั้นที่ 1 : คำนวณซูโดอินเวอร์ส (Pseudo Inverse) เมื่อ $\mathbf{X}^+ = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$

กำหนดให้

$$\mathbf{X} = \begin{bmatrix} 1 & 22 \\ 1 & 28 \\ 1 & 32 \\ 1 & 33 \\ 1 & 43 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 1800 \\ 2900 \\ 3400 \\ 4400 \\ 5100 \end{bmatrix} \quad \mathbf{X}^T = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix}$$

จะได้

$$\mathbf{X}^T \mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix} \times \begin{bmatrix} 1 & 22 \\ 1 & 28 \\ 1 & 32 \\ 1 & 33 \\ 1 & 43 \end{bmatrix} = \begin{bmatrix} 5 & 158 \\ 158 & 5230 \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

$$(\mathbf{X}^T \mathbf{X})^{-1} = \frac{1}{1186} \begin{bmatrix} 5230 & -158 \\ -158 & 5 \end{bmatrix} = \begin{bmatrix} 4.40978 & -0.13322 \\ -0.13322 & 0.00422 \end{bmatrix}$$

$$\begin{aligned} (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T &= \begin{bmatrix} 4.40978 & -0.13322 \\ -0.13322 & 0.00422 \end{bmatrix} \times \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 22 & 28 & 32 & 33 & 43 \end{bmatrix} \\ &= \begin{bmatrix} 1.47892 & 0.67959 & 0.14671 & 0.01349 & -1.3187 \\ -0.04047 & -0.01517 & 0.00168 & 0.00590 & 0.04806 \end{bmatrix} \end{aligned}$$

ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ เมื่อ $\beta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$

$$(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = \begin{bmatrix} 1.47892 & 0.67959 & 0.14671 & 0.01349 & -1.3187 \\ -0.04047 & -0.01517 & 0.00168 & 0.00590 & 0.04806 \end{bmatrix} \times \begin{bmatrix} 1800 \\ 2900 \\ 3400 \\ 4400 \\ 5100 \end{bmatrix}$$
$$= \begin{bmatrix} -1534.40 \\ 159.95 \end{bmatrix}$$

ขั้นที่ 3 : นำ β มาสร้างสมการเชิงเส้น ได้ดังนี้

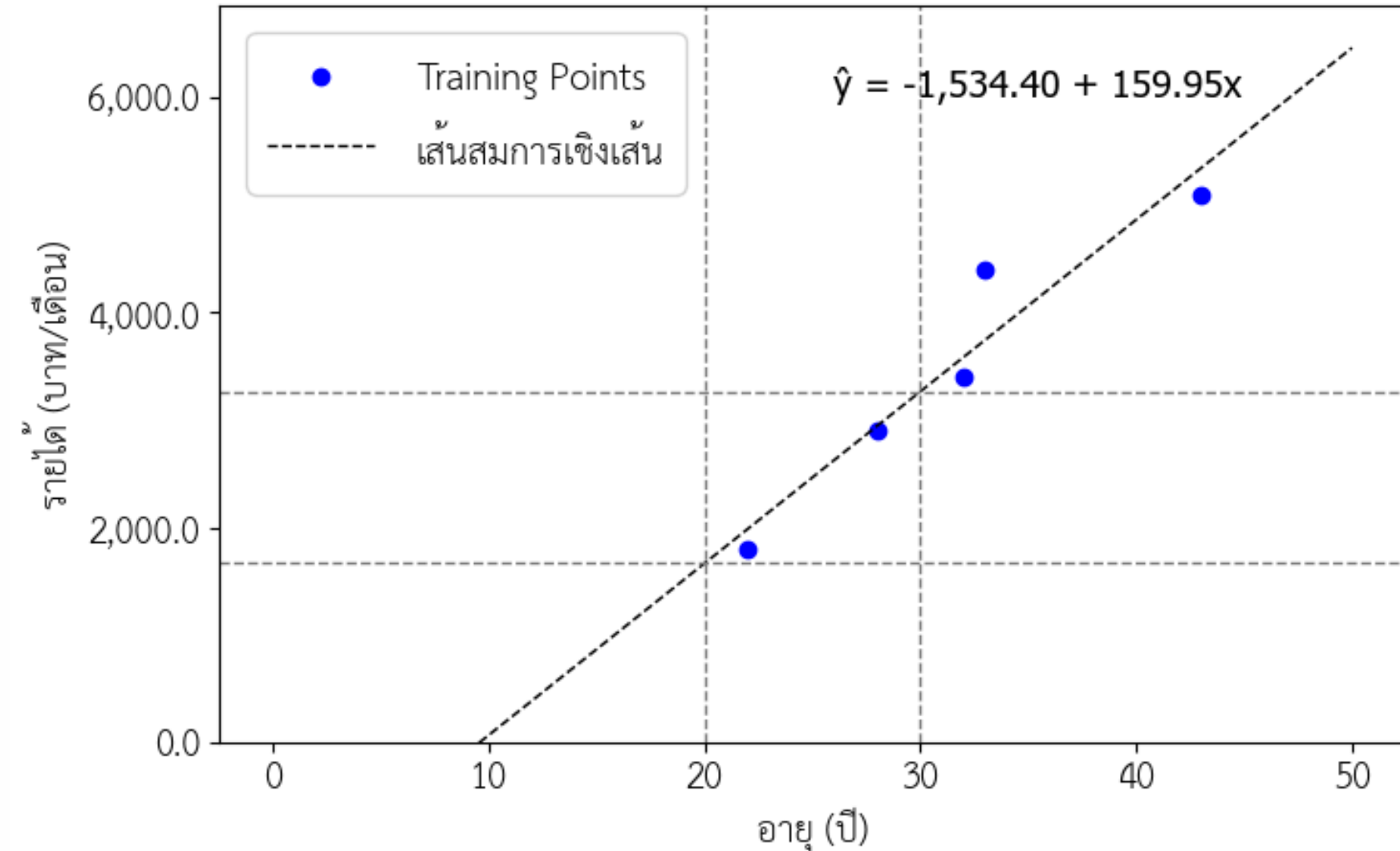
$$\hat{y} = -1,534.40 + 159.95x_1$$

เมื่อ $x_1 = 20$

$$\hat{y} = -1534.40 + 159.95 \times 20 = 1,664.6$$

เมื่อ $x_1 = 30$

$$\hat{y} = -1534.40 + 159.95 \times 30 = 3,264.1$$



9.3.3 ตัวอย่างโค้ด OLS

- เขียนโค้ดแบบไม่ใช่ไลบรารี ML

```
1 #Import Python Library
2 import numpy as np
3 from numpy.linalg import inv
4
5 #กำหนดค่าเริ่มต้น
6 x_train = np.array([[1,22], [1,28], [1,32], [1,33], [1,43]])
7 y_train = np.array([1800, 2900, 3400, 4400, 5100])
8 x_test = np.array([20, 30])
9
10 #ขั้นที่ 1 : คำนวณซุโดอินเวอร์ส (Pseudo Inverse) เมื่อ  $X^+ = [(X^T X)]^{-1} X^T$ 
11 mat_trans = np.transpose(x_train) #  $x^T$ 
12 mat_mul = np.matmul(mat_trans, x_train) #  $X^T X$ 
13 inverse = np.linalg.inv(mat_mul) #  $[(X^T X)]^{-1}$ 
14 pseudo_inv = np.matmul(inverse, mat_trans) #  $(x^T x)^{-1} * x^T$ 
15 print(f'ขั้นที่ 1 : ซุโดอินเวอร์ส คือ {pseudo_inv}')
16
17 #ขั้นที่ 2 : คำนวณค่าสัมประสิทธิ์ เมื่อ  $\beta = [(X^T X)]^{-1} X^T y$ 
18 beta = np.matmul(pseudo_inv, y_train)
19 print(f'ขั้นที่ 2 : ค่าสัมประสิทธิ์ คือ {beta}')
20
21 #ขั้นที่ 3 : สร้างสมการเชิงเส้นเพื่อทำนาย
22 print(f'ขั้นที่ 3 : สมการเชิงเส้น คือ  $y = \{round(beta[0], 2)\} + \{round(beta[1], 2)\}x$ ')
23 for x in x_test:
24     print(f'\tเมื่ออายุ {x} ปี รายได้จากการประเมิน คือ {(beta[0] + beta[1] * x):,.2f} บาท/เดือน')
```

```
ขั้นที่ 1 : ซุโดอินเวอร์ส คือ [[ 1.47892074  0.67959528  0.14671164  0.01349073 -1.31871838]
[-0.04047218 -0.01517707  0.00168634  0.00590219  0.04806071]]
ขั้นที่ 2 : ค่าสัมประสิทธิ์ คือ [-1534.40134907  159.94940978]
ขั้นที่ 3 : สมการเชิงเส้น คือ  $y = -1534.4 + 159.95x$ 
          เมื่ออายุ 20 ปี รายได้จากการประเมิน คือ 1,664.59 บาท/เดือน
          เมื่ออายุ 30 ปี รายได้จากการประเมิน คือ 3,264.08 บาท/เดือน
```

- เขียนโค้ดแบบใช้ไลบรารี Scikit-learn

```
1 #Import Python Library
2 import numpy as np
3 from sklearn import linear_model
4
5 #Load Data
6 x_train = np.array([22, 28, 32, 33, 43]).reshape(-1, 1)
7 y_train = np.array([1800, 2900, 3400, 4400, 5100])
8 x_test = np.array([20, 30]).reshape(-1, 1)
9
10 #Build Model
11 net=linear_model.LinearRegression()
12 net.fit(x_train, y_train)
13
14 #Test Model
15 y_pred=net.predict(x_test)
16 #print(f'สมการเชิงเส้น คือ y={net.intercept_:,.2f} + {net.coef_[0]:,.2f}x')
17 for i, x in enumerate(x_test):
18     print(f'\tเมื่ออายุ {x[0]} ปี รายได้จากการประเมิน คือ {y_pred[i]:,.2f} บาท/เดือน')
```

เมื่ออายุ 20 ปี รายได้จากการประเมิน คือ 1,664.59 บาท/เดือน

เมื่ออายุ 30 ปี รายได้จากการประเมิน คือ 3,264.08 บาท/เดือน

9.4 Decision Tree (DT)

- อาศัยแนวคิดของความน่าจะเป็นเพื่อสร้างต้นไม้ตัดสินใจ
- ไม่ซับซ้อนสามารถเข้าใจได้ง่าย
- ไม่ Sensitive ต่อ Outliers
- เทคนิค Decision Tree มีดังนี้
 - ID3 (Iterative Dichotomiser 3) : ใช้สำหรับตัวแปรต้นแบบ Categorical ในงาน Classification เท่านั้น
 - C4.5 และ C5.0 : ใช้สำหรับตัวแปรต้นทั้งแบบ Categorical และ Numerical ในงาน Classification เท่านั้น
 - CART, Random Forest, และ XGBoost : ใช้สำหรับตัวแปรต้นทั้งแบบ Categorical และ Numerical ในงาน Classification และ Regression

Algorithm	ประเภทข้อมูล (Input)	ประเภทงาน (Output)	จุดเด่นเพิ่มเติม
ID3	Categorical เท่านั้น	Classification	ใช้ค่า Entropy และ Information Gain ในการเลือก Node
C4.5 / C5.0	Categorical / Numerical	Classification	ใช้ค่า Gain Ratio แทน Information Gain รองรับข้อมูลต่อเนื่องและจัดการ Missing Value ได้
CART	Categorical / Numerical	Classification / Regression	ใช้ Gini Index (Classification) และ MSE (Regression)
Random Forest	Categorical / Numerical	Classification / Regression	เป็น Ensemble Method (Bagging) เพื่อลดอาการ Overfitting
XGBoost	Categorical / Numerical	Classification / Regression	เป็น Ensemble Method (Boosting)

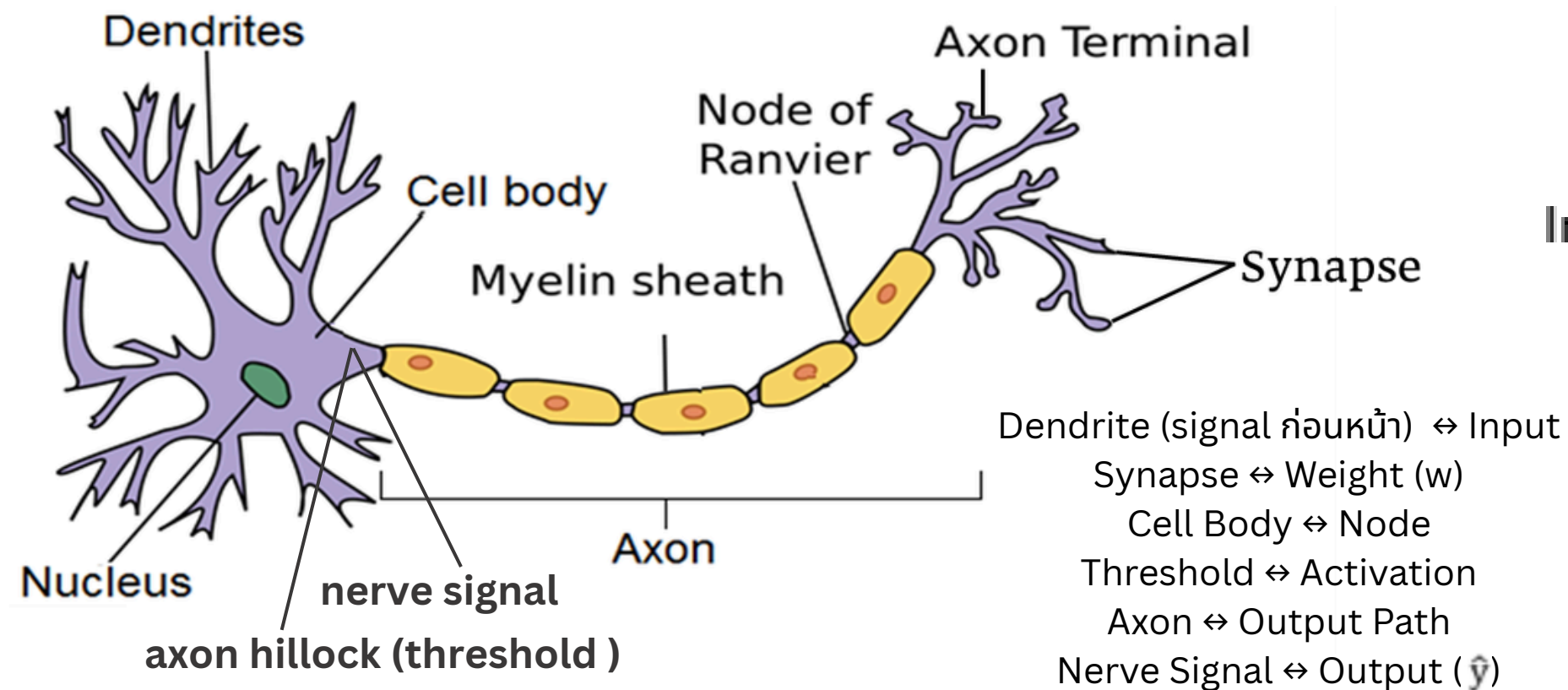
9.4.1 ตัวอย่างโค้ด CART (Regression)

```
1 #Import Python library
2 from sklearn.tree import DecisionTreeRegressor
3
4 #Load Data
5 x_train = [[50, 5],[100, 4], [80, 5], [200, 3], [500, 1], [300, 2], [700, 0], [600, 1], [250, 3],[120, 4]]
6 y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
7 x_test = [220, 2]
8
9 #Build Model
10 net = DecisionTreeRegressor()
11 net.fit(x_train,y_train)
12
13 #Test Model
14 y_pred=net.predict([x_test])*1000
15 print(f'ราคาประเมินที่ดิน คือ {y_pred:.2f} บาท/ตร.ว.')
```

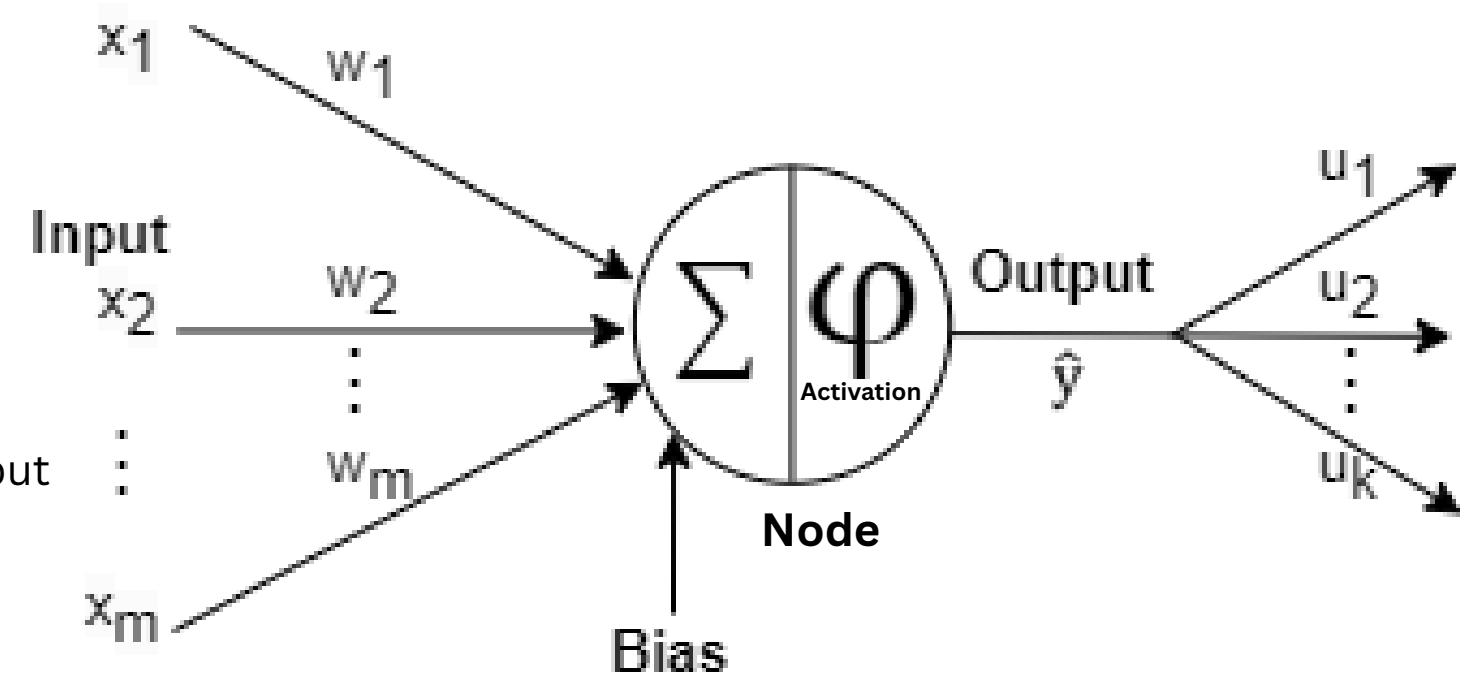
ราคาประเมินที่ดิน คือ 190,000.00 บาท/ตร.ว.

9.5 Artificial Neural Network (ANN)

โครงข่ายประสาทเทียม (Artificial Neural Network: ANN) คือ โครงข่ายประสาทเทียมที่สามารถเลียนแบบการทำงานของสมองมนุษย์ ใช้ในการประมวลผลข้อมูลและเรียนรู้รูปแบบ (patterns) จากข้อมูลที่ป้อนเข้าไป



Biological Neural Network

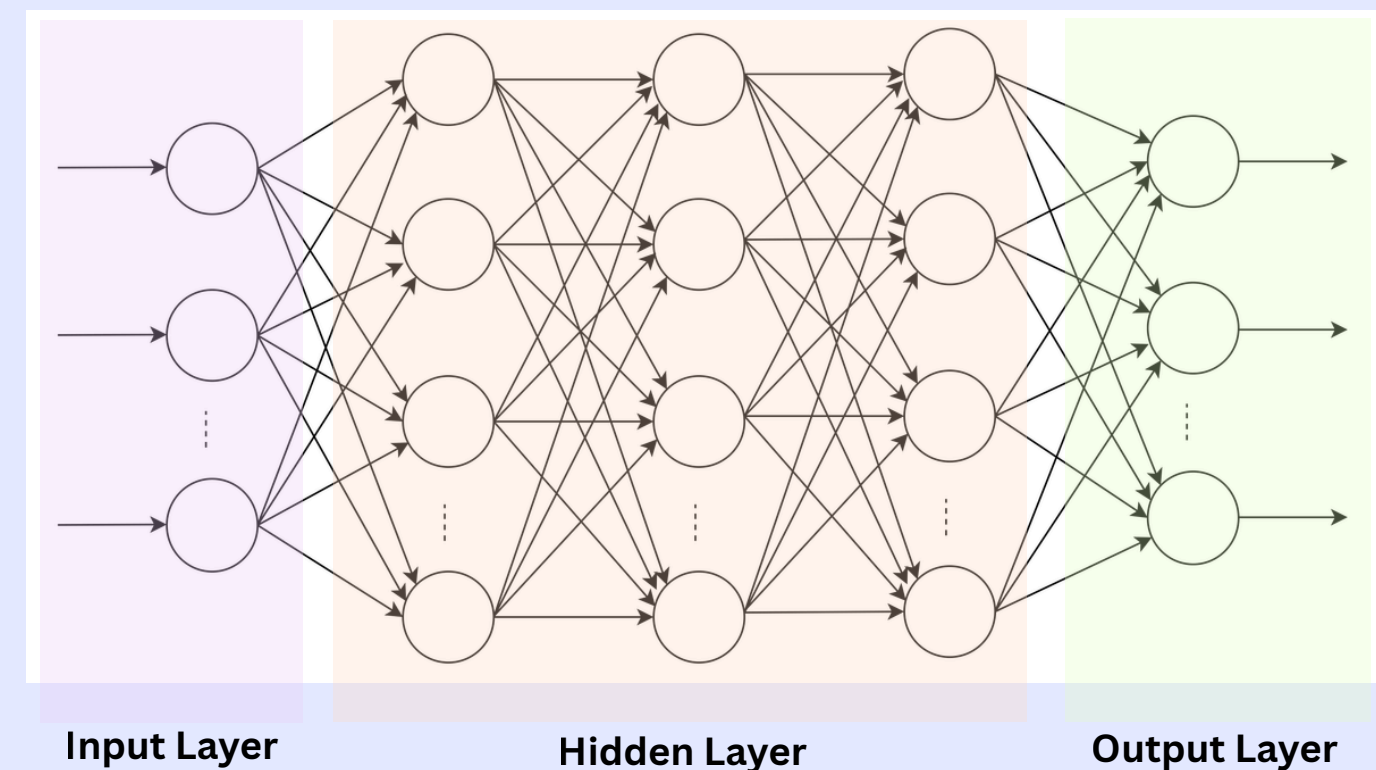


Artificial Neural Network (ANN)

ภาพที่ 9-9 : เปรียบเทียบ Biological Neural Network กับ Artificial Neural Network (ANN)

9.5.1 Multilayer Perceptron (MLP)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Feed forward neural network
 - รับข้อมูลเข้าในทิศทางจาก Input → Hidden Layer(s) → Output
 - ใน output layer, คำนวณผลลัพธ์ (predicted) แล้วนำไปเทียบกับค่าจริง (actual) เพื่อคำนวณ Error
- เรียนรู้โดยใช้ Backpropagation
 - ย้อนกลับไปปรับ weight ใหม่ (อัปเดตความรู้) โดยใช้ error ที่คำนวณได้
- เป็น Universal Approximator
 - ในปี 1989 George Cybenko ได้พิสูจน์ว่า ANN ที่มีเพียง 1 Hidden Layer และใช้ Sigmoid เป็น Activation Function สามารถประมาณฟังก์ชันต่อเนื่องได้ทุกชนิด



9.5.1.1 ตัวอย่างการคำนวณ MLP

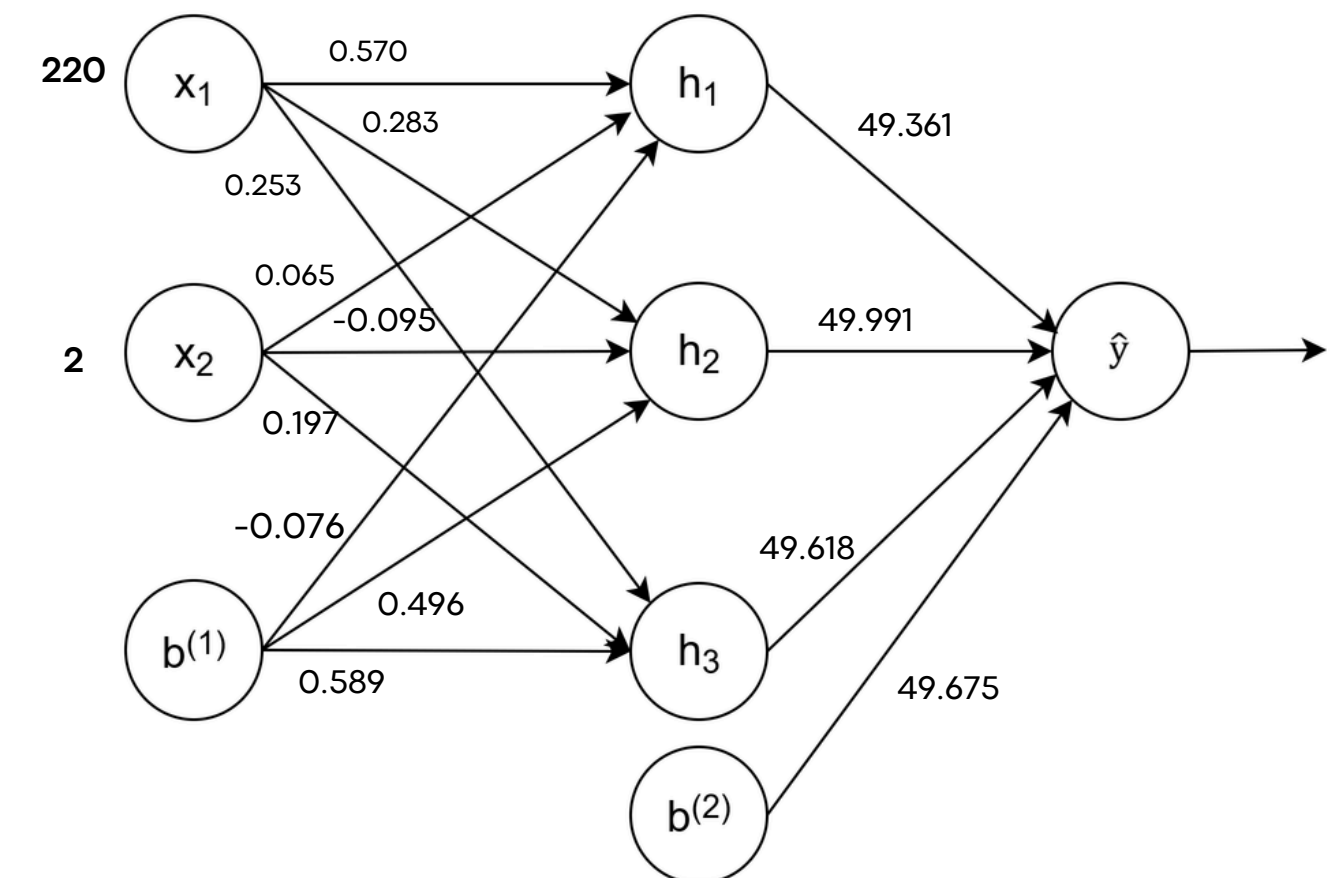
จงแสดงวิธีการคำนวณ MLP เพื่อประเมินราคาที่ดิน

กำหนดให้

- Input Node = 2
- Hidden Node = 3 (Logistic Sigmoid)
- Output Node = 1 (Linear)
- Hidden Weights (Input → Hidden): $\mathbf{W}_h = \begin{bmatrix} 0.570 & 0.283 & 0.253 \\ 0.065 & -0.095 & 0.197 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [-0.076, 0.496, 0.589]$
- Output Weight (Hidden → Output): $\mathbf{W}_y = \begin{bmatrix} 49.361 \\ 49.991 \\ 49.618 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [49.675]$

ตารางที่ 9-3 : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ระยะทางถึงถนน/ รถไฟฟ้า (เมตร)	จำนวนสถานที่สำคัญ ในรัศมี 200 ม. (แหล่ง)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	220	2	?



วิธีทำ

- คำนวณใน Hidden Layer

$$h_1 = \varphi(0.570 \times 220 + 0.065 \times 2 - 0.076) = \varphi(125.454)$$
$$= \frac{1}{1 + e^{-125.454}} = 1$$

$$h_2 = \varphi(0.283 \times 220 + (-0.095) \times 2 + 0.496) = \varphi(62.566)$$
$$= \frac{1}{1 + e^{-62.566}} = 1$$

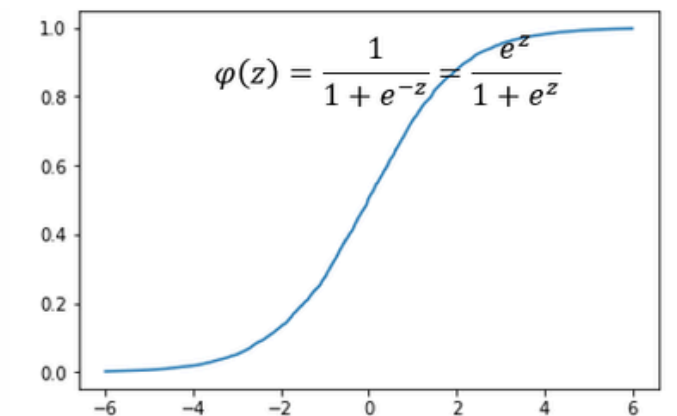
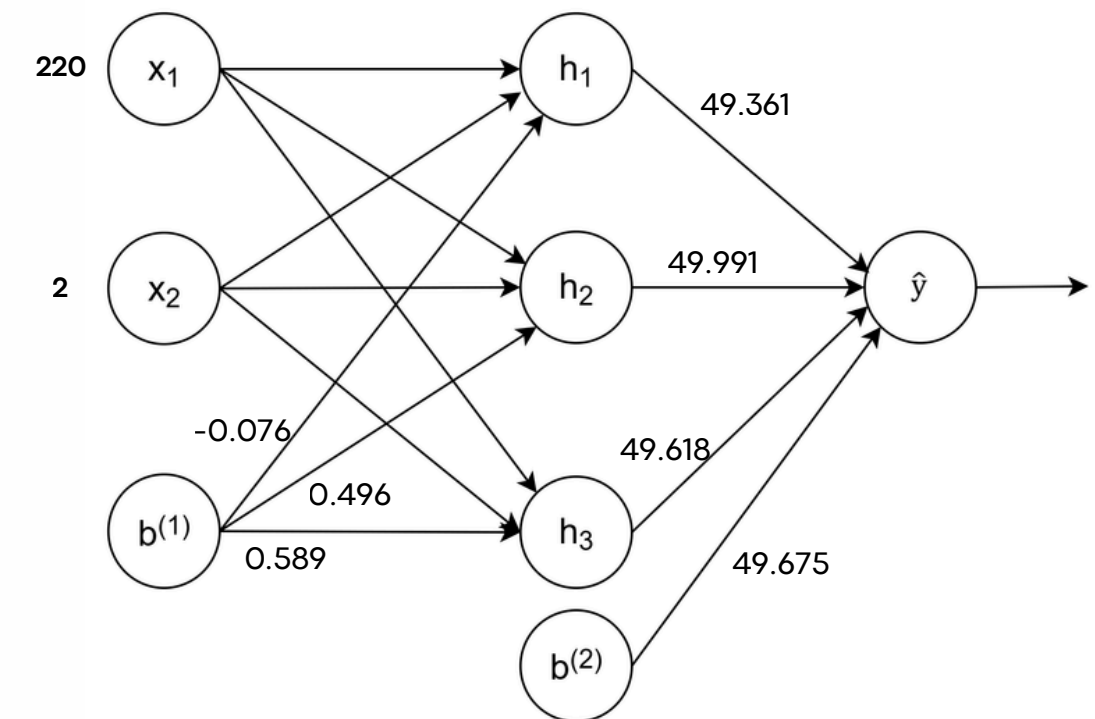
$$h_3 = \varphi(0.253 \times 220 + (0.197) \times 2 + 0.589) = \varphi(56.643)$$
$$= \frac{1}{1 + e^{-56.643}} = 1$$

- คำนวณใน Output Layer

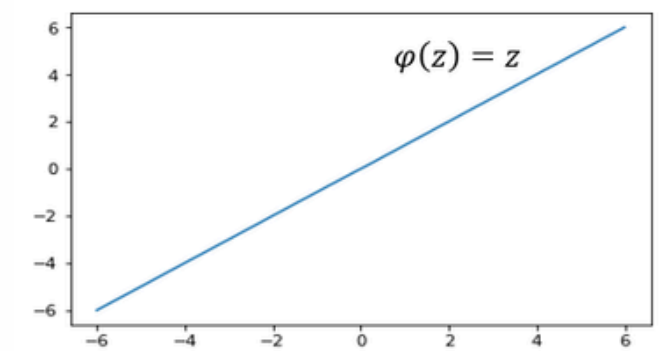
$$\hat{y} = \varphi(49.361 \times 1 + 49.991 \times 1 + 49.618 \times 1 + 49.675) = \varphi(198.645) = 198.645$$

- คูณ 1,000 เพื่อปรับให้อยู่ในหน่วย บาท/ตร.ว. จะได้
 $198.645 \times 1,000 = 198,645$

ตอบ ราคาที่ดินที่ประเมินได้ คือ 198,645 บาท/ตร.ว.



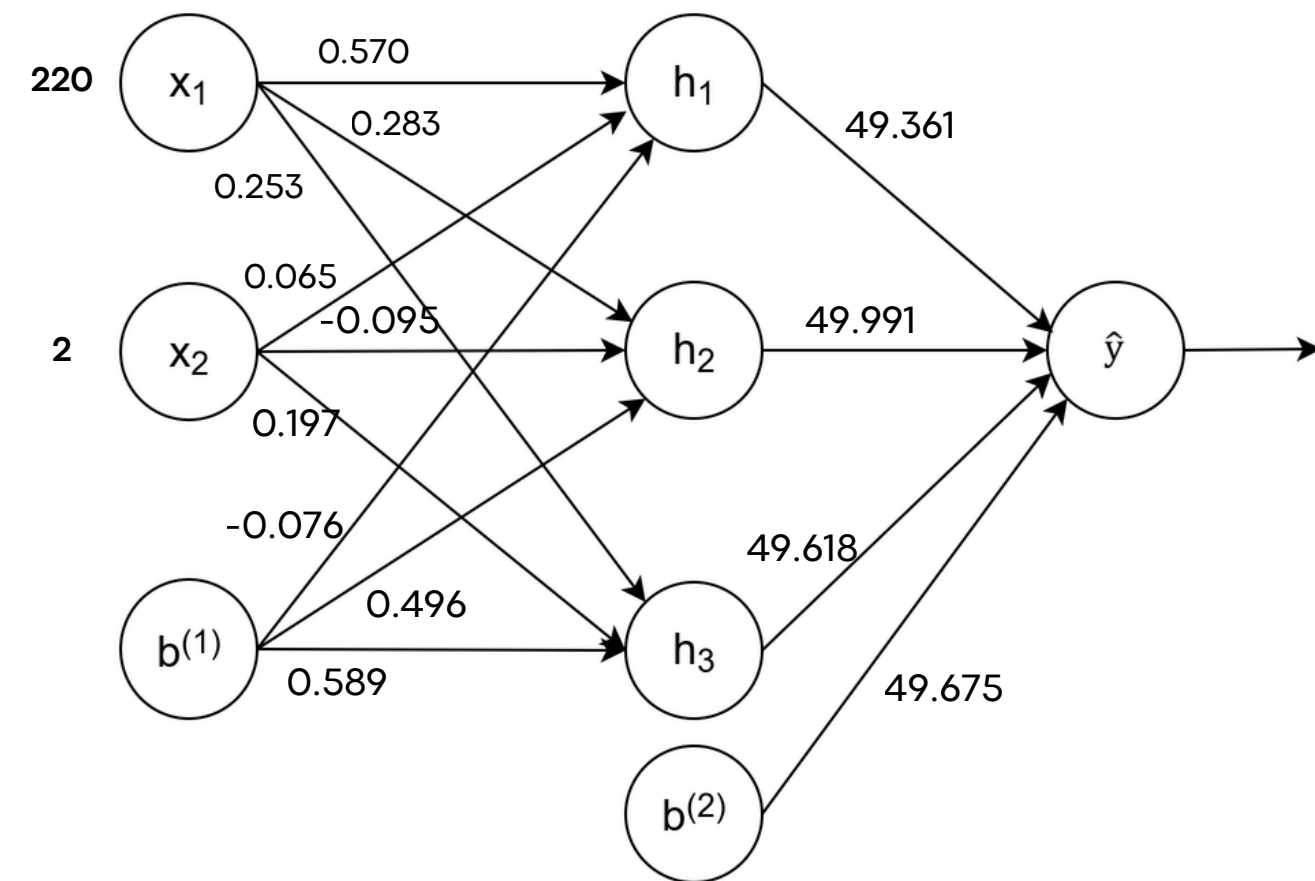
Logistic Sigmoid (logistic)



Identity (linear)

9.5.1.2 ตัวอย่างโค้ด MLP

```
1  #Import Python Library
2  from sklearn.neural_network import MLPRegressor
3
4  #Load Data
5  x_train = [[50, 5],[100, 4], [80, 5], [200, 3], [500, 1],
6             [300, 2], [700, 0], [600, 1], [250, 3],[120, 4]]
7  y_train = [250, 240, 260, 200, 150, 180, 130, 140, 190, 230]
8  x_test = [[220, 2]]
9
10 #Build Model
11 net = MLPRegressor(
12     hidden_layer_sizes=(3),
13     activation='logistic',#sigmoid function
14     solver='sgd',
15     batch_size=5,
16     learning_rate_init=0.001,
17     max_iter=250,
18     random_state=0)
19
20 net.fit(x_train,y_train)
21 print(f'weight: {net.coefs_}') #show weight
22 print(f'bias: {net.intercepts_}') #show bias
23
24 #Test Model
25 y_pred = net.predict(x_test)
26 print(f'ราคาประเมิน คือ {y_pred[0]*1000:,.0f} บาท/ตร.ว.')
```



```
weight: [array([[ 0.56959927,  0.28340219,  0.25298845],
                [ 0.06548977, -0.09544737,  0.19674558]]), array([[49.36103808],
                [49.99053636],
                [49.61814864]])]
bias: [array([-0.07585905,  0.49578219,  0.58893365]), array([49.67477853])]
ราคาประเมิน คือ 198,645 บาท/ตร.ว.
```

9.5.2 Recurrent Neural Network (RNN)

- เป็นรูปแบบหนึ่งของโครงข่ายประสาทเทียม (ANN)
- เป็น Recurrent neural network
รับข้อมูลเข้ามาแบบลำดับ (Sequential Data) เช่น ข้อความ เสียง หรือสัญญาณเวลา โดยส่งข้อมูลผ่านชั้นต่าง ๆ เช่น Recurrent Layer (เช่น Simple RNN, LSTM, GRU) และ Fully Connected Layer
- เรียนรู้โดยใช้ Backpropagation Through Time (BPTT)
เป็นการปรับปรุงความรู้หรือน้ำหนัก (Weight) ย้อนกลับตามลำดับเวลา โดยใช้ค่า error จาก output แต่ละช่วงเวลามาคำนวณย้อนกลับ
- มีคุณสมบัติในการจดจำลำดับและบริบท (Sequence and Context Memory)
สามารถเก็บข้อมูลจากขั้นตอนก่อนหน้า (Previous Steps) เพื่อใช้ในการประมวลผลขั้นตอนถัดไป ทำให้เข้าใจความสัมพันธ์ในลำดับข้อมูลได้
- นิยมใช้ในงานประมวลผลข้อมูลลำดับ (Sequential Data Processing)
เช่น Time Series Forecasting, Natural Language Processing (NLP), Machine Translation

9.5.2.1 ตัวอย่างการคำนวณ RNN (Elman Network)

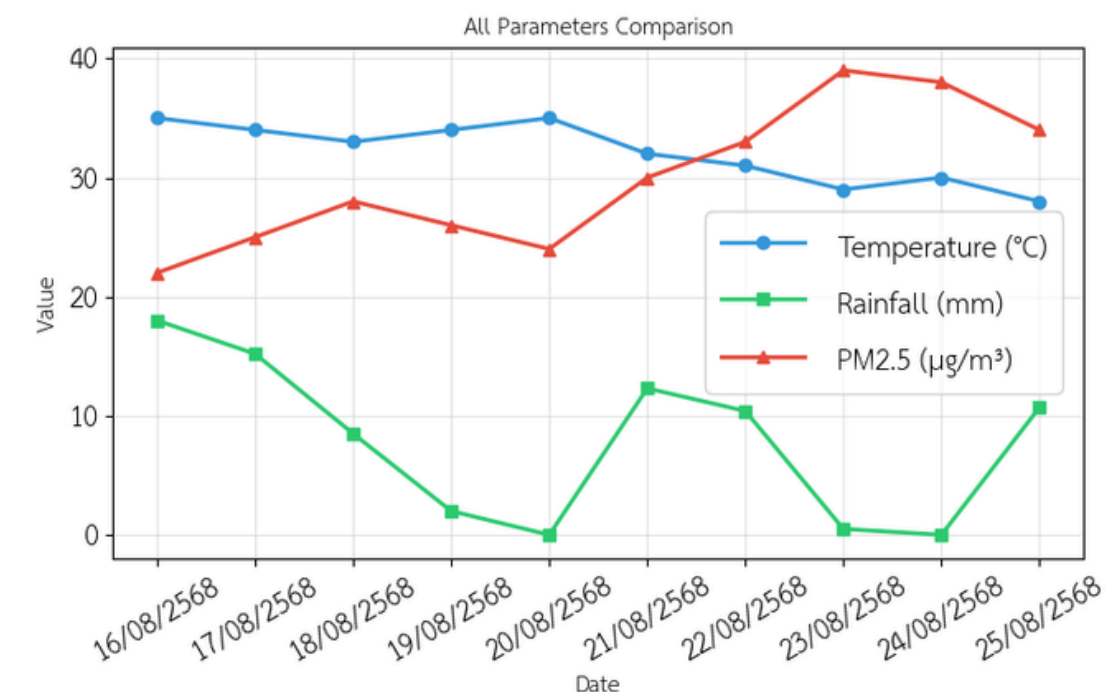
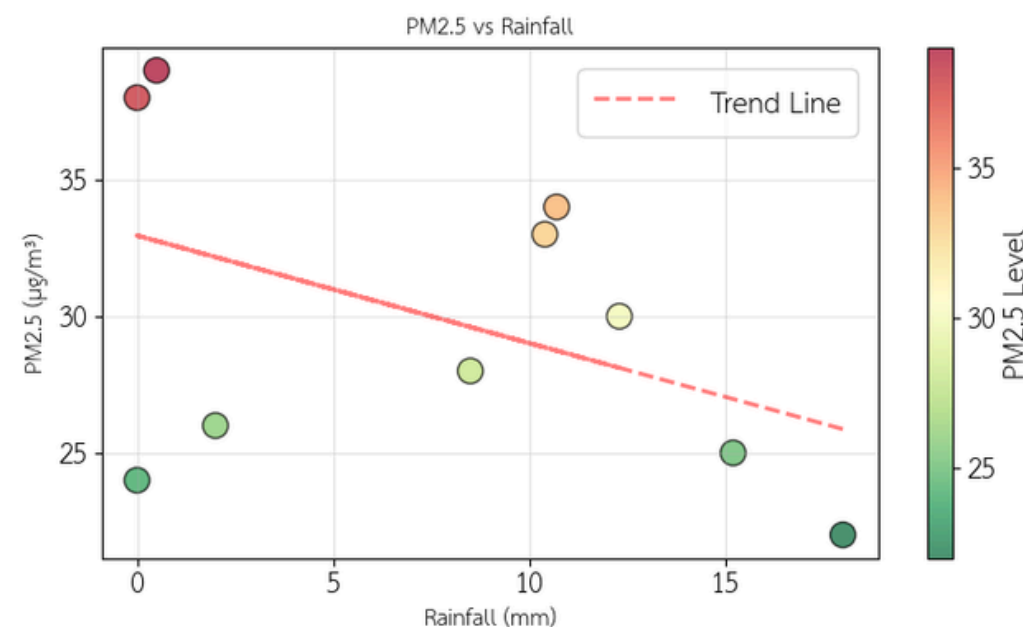
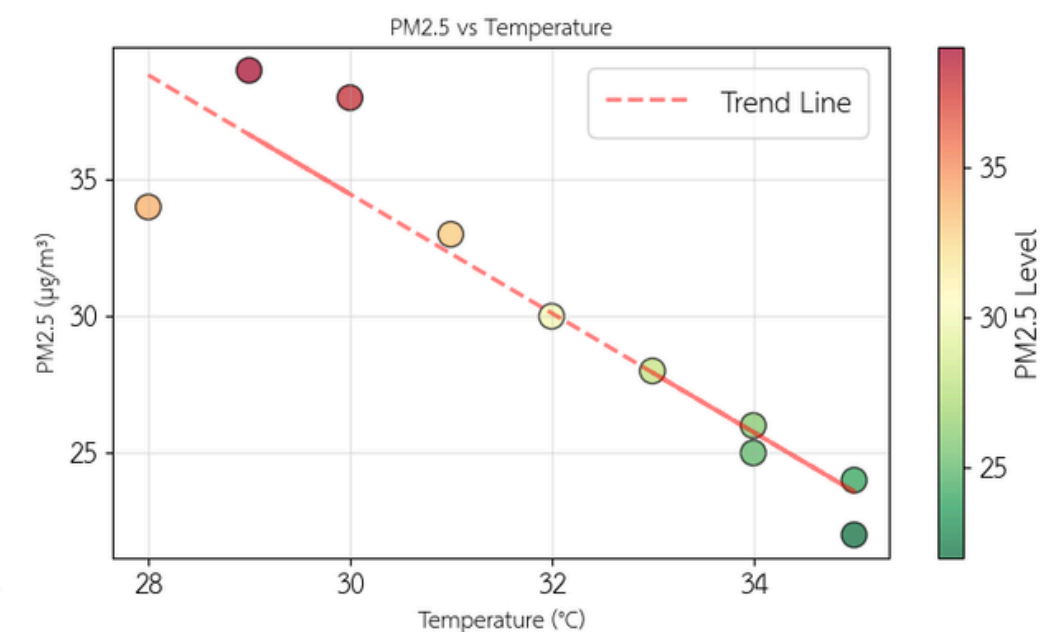
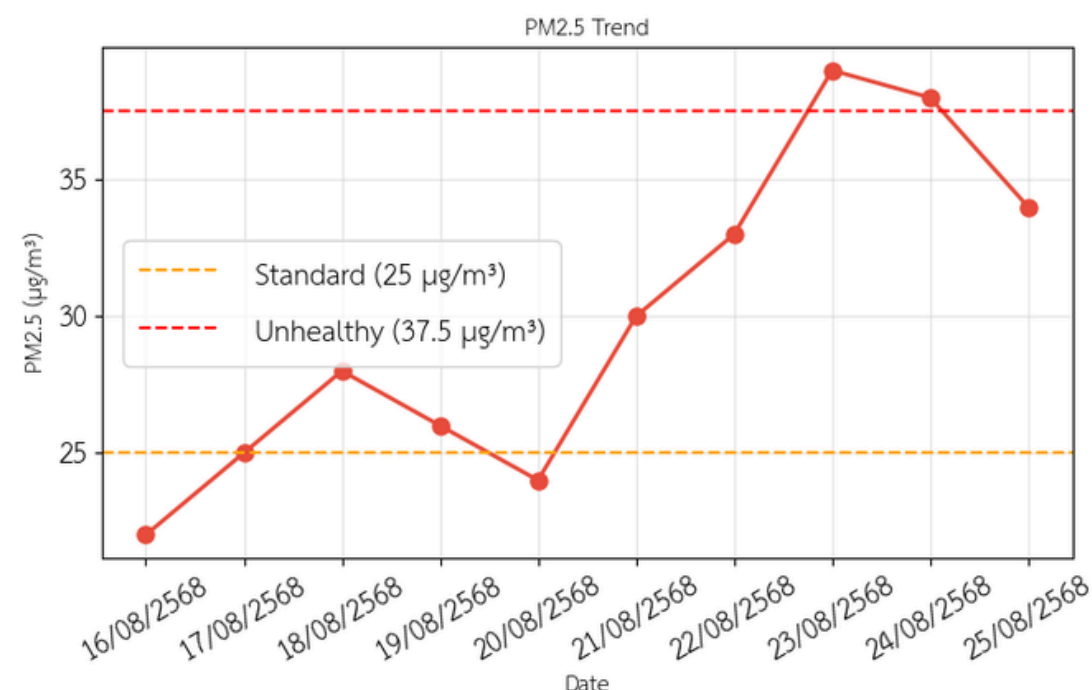
จงแสดงวิธีการคำนวณ RNN (Elman Network) เพื่อทำนายปริมาณ PM2.5 ณ วันที่ 19/8/2568

กำหนดให้

- Input Node = 3,
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (ReLU)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [-0.05, 0.08, 0.1]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.2$
- Hidden Bias: $\mathbf{b}_h = 0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 20$
- Output Bias: $\mathbf{b}_y = 7$

ตารางที่ 9-4 : ชุดข้อมูล PM2.5

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 ($\mu\text{g}/\text{m}^3$)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28
19/8/2568	34	2	26
20/8/2568	35	0	24
21/8/2568	32	12.3	30
22/8/2568	31	10.4	33
23/8/2568	29	0.5	39
24/8/2568	30	0	38
25/8/2568	28	10.7	34



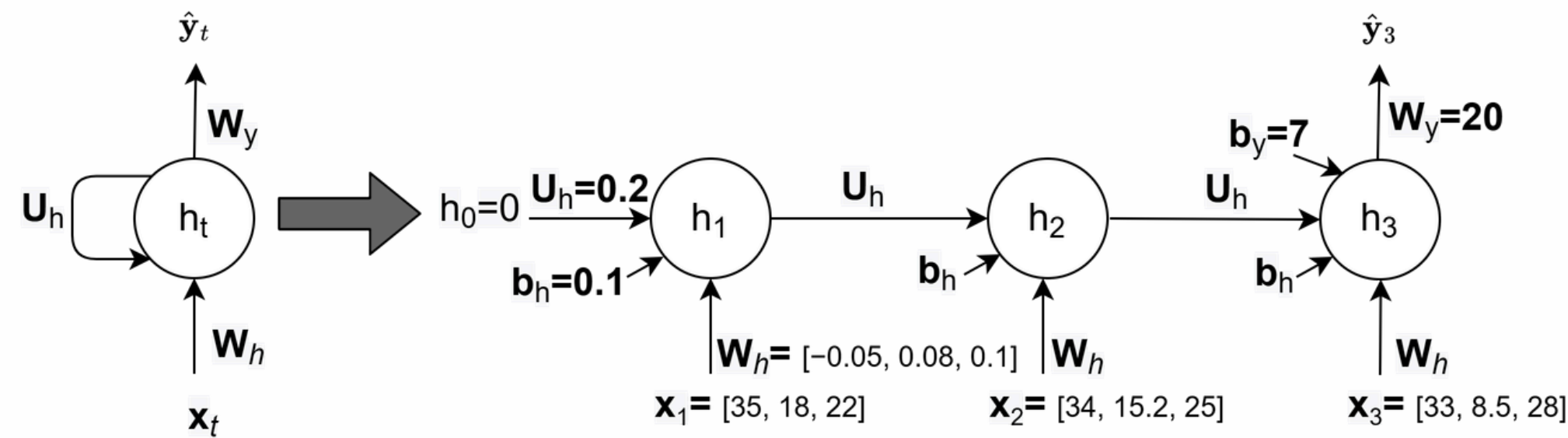
ภาพที่ 9-10 : ชุดข้อมูล PM2.5

วิธีทำ

กำหนดให้

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$



ค่า $\mathbf{W}_h$, $\mathbf{U}_h$, และ $\mathbf{b}_h$ จะเท่าเดิม

	วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)
$\mathbf{x}_1$	16/8/2568	35	18	22
$\mathbf{x}_2$	17/8/2568	34	15.2	25
$\mathbf{x}_3$	18/8/2568	33	8.5	28

- คำนวณใน Hidden Layer (State)

$$\mathbf{h}_t = \varphi_h(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h \mathbf{h}_{t-1} + \mathbf{b}_h)$$

เมื่อ $t=1$ (16/08/2568)

$$Z_1 = -0.05 \times 35 + 0.08 \times 18 + 0.1 \times 22 + 0.2 \times 0 + 0.1 = 1.99$$

$$\mathbf{h}_1 = \frac{1}{1 + e^{-1.99}} = 0.88$$

เมื่อ $t=2$ (17/08/2568)

$$Z_2 = -0.05 \times 34 + 0.08 \times 15.2 + 0.1 \times 25 + 0.2 \times 0.88 + 0.1 = 2.292$$

$$\mathbf{h}_2 = \frac{1}{1 + e^{-2.292}} = 0.908$$

เมื่อ $t=3$ (18/08/2568)

$$Z_3 = -0.05 \times 33 + 0.08 \times 8.5 + 0.1 \times 28 + 0.2 \times 0.908 + 0.1 = 2.112$$

$$\mathbf{h}_3 = \frac{1}{1 + e^{-2.112}} = 0.892$$

เมื่อ $\mathbf{W}_h = [-0.05, 0.08, 0.1]$

$\mathbf{U}_h = 0.2$ $\mathbf{h}_0 = 0$ $\mathbf{b}_h = 0.1$

$\mathbf{x}_1 = [35, 18, 22]$

$\mathbf{x}_2 = [34, 15.2, 25]$

$\mathbf{x}_3 = [33, 8.5, 28]$

วัน/เดือน/ปี	อุณหภูมิ (°C)	ปริมาณน้ำฝน (mm)	PM 2.5 (µg/m³)
16/8/2568	35	18	22
17/8/2568	34	15.2	25
18/8/2568	33	8.5	28

- **คำนวณใน Output Layer**

$$\hat{\mathbf{y}}_t = \varphi_y(\mathbf{W}_y \mathbf{h}_t + \mathbf{b}_y)$$

$$\hat{\mathbf{y}}_3 = \text{ผลการทำนายของวันที่ 19/08/2568}$$

$$\hat{\mathbf{y}}_3 = \varphi_y(20 \times 0.892 + 7) = \varphi_y(24.84)$$

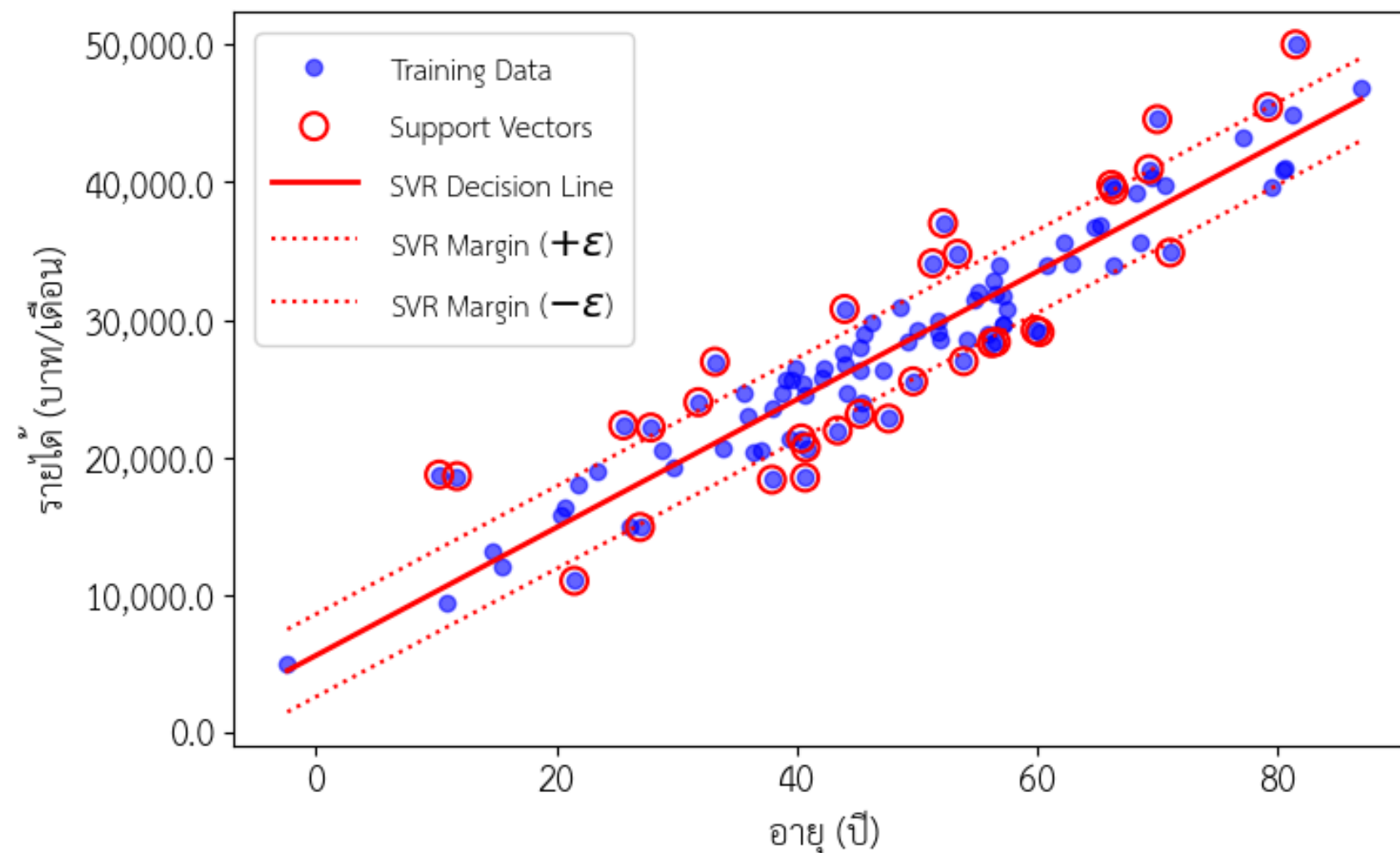
$$\hat{\mathbf{y}}_3 = \max(0, 24.84) = 24.84$$

เมื่อ $\mathbf{W}_y = 20$
 $\mathbf{h}_3 = 0.892$
 $\mathbf{b}_y = 7$

ตอบ ผลการทำนายปริมาณ PM2.5 ณ วันที่ 19/8/2568 คือ $24.84 \mu\text{g}/\text{m}^3$

9.6 Support Vector Regression (SVR)

- อาศัยแนวคิดของ Margin Maximization เพื่อหา Optimal Hyperplane (สมการพหุคูณ) ซึ่งทำให้ข้อมูลส่วนใหญ่อยู่ภายในช่วงความคลาดเคลื่อนที่ยอมรับได้ (ϵ -Insensitive Tube)
- จุดข้อมูลที่อยู่บนขอบหรือนอกขอบของช่วง ϵ จะเรียกว่า Support Vectors ซึ่งเป็นจุดที่มีผลต่อการกำหนด Optimal Hyperplane โดยตรง
- จัดการกับข้อมูลที่ไม่เป็นเชิงเส้นผ่าน Kernel Function



ภาพที่ 9-11 : ตัวอย่างการประมาณการรายได้ด้วย SVR

9.6.1 การคำนวณสำหรับ SVR

ขั้นตอนการคำนวณ

- การเทรนโมเดล

- กำหนดค่าเริ่มต้น

$x \in \mathbb{R}^m$, where m denotes the total number of features. $y \in \mathbb{R}$

- คำนวณ Kernel Function

$K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$ เมื่อใช้ Linear เป็น Kernel Function จะได้ $K(x_i, x_j) = x_i^T x_j$

- คำนวณหา Alpha : SVR จะมีตัวแปร Lagrange multipliers สองตัวคือ α_i และ α_i^* เพื่อจัดการขอบเขตบนและล่าง

$$\max_{\alpha, \alpha^*} \sum_{i=1}^n y_i (\alpha_i - \alpha_i^*) - \epsilon \sum_{i=1}^n (\alpha_i + \alpha_i^*) - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n (\alpha_i - \alpha_i^*) (\alpha_j - \alpha_j^*) K(x_i, x_j)$$

$$\text{subject to } \sum_{i=1}^n (\alpha_i - \alpha_i^*) = 0$$

$$0 \leq \alpha_i, \alpha_i^* \leq C \text{ โดยปกติ } C \text{ จะมีค่าเท่ากับ } 1$$

- คำนวณ Weight (ใช้เฉพาะ Linear)

$$w = \sum_{i=1}^n (\alpha_i - \alpha_i^*) \phi(x_i)$$

- คำนวณ Bias

เลือก x_k ที่เป็น Support Vector (ตัวที่อยู่บนขอบต่อพอดี) มา 1 ตัว ที่มีค่า $\alpha_i > 0$ เพื่อมาคำนวณหา Bias

$$b = y_k - \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(x_i, x_k) - \epsilon$$

- **การทดสอบโมเดล**

ใช้ Decision Function ในการตัดสินใจ (ทำนาย)

$$\hat{y} = \sum_{i=1}^n (\alpha_i - \alpha_i^*) K(x_i, x) + b$$

เมื่อใช้ Linear เป็น Kernel Function จะได้

$$\hat{y} = w^T x + b$$

9.6.2 ตัวอย่างการคำนวณ SVR

จงแสดงวิธีการคำนวณ Support Vector Regression เพื่อประเมินราคาที่ดิน โดยให้ใช้ค่า และชุดข้อมูลทดสอบต่อไปนี้

- weight (w) = $[-0.1833]$
- bias (b) = 248.33

ตารางที่ 9-5 : ชุดข้อมูลทดสอบ (Test Set)

รายการ	ระยะทางถึงถนน/รถไฟฟ้า (เมตร)	ราคาที่ดิน (1,000 บาท/ตร.ว.)
1	220	?

วิธีทำ

จาก

$$\hat{y} = w^T x + b$$

$$w = [-0.1833] \quad b = 248.33$$

จะได้

$$\begin{aligned}\hat{y} &= -0.1833 \times 220 + 248.33 \\ &= 208.004\end{aligned}$$

คูณ 1,000 เพื่อปรับให้อยู่ในหน่วย บาท/ตร.ว.

$$\text{จะได้ } 208.004 \times 1,000 = 208,004$$

ตอบ ราคาที่ดินที่ประเมินได้ คือ 208,004 บาท/ตร.ว.

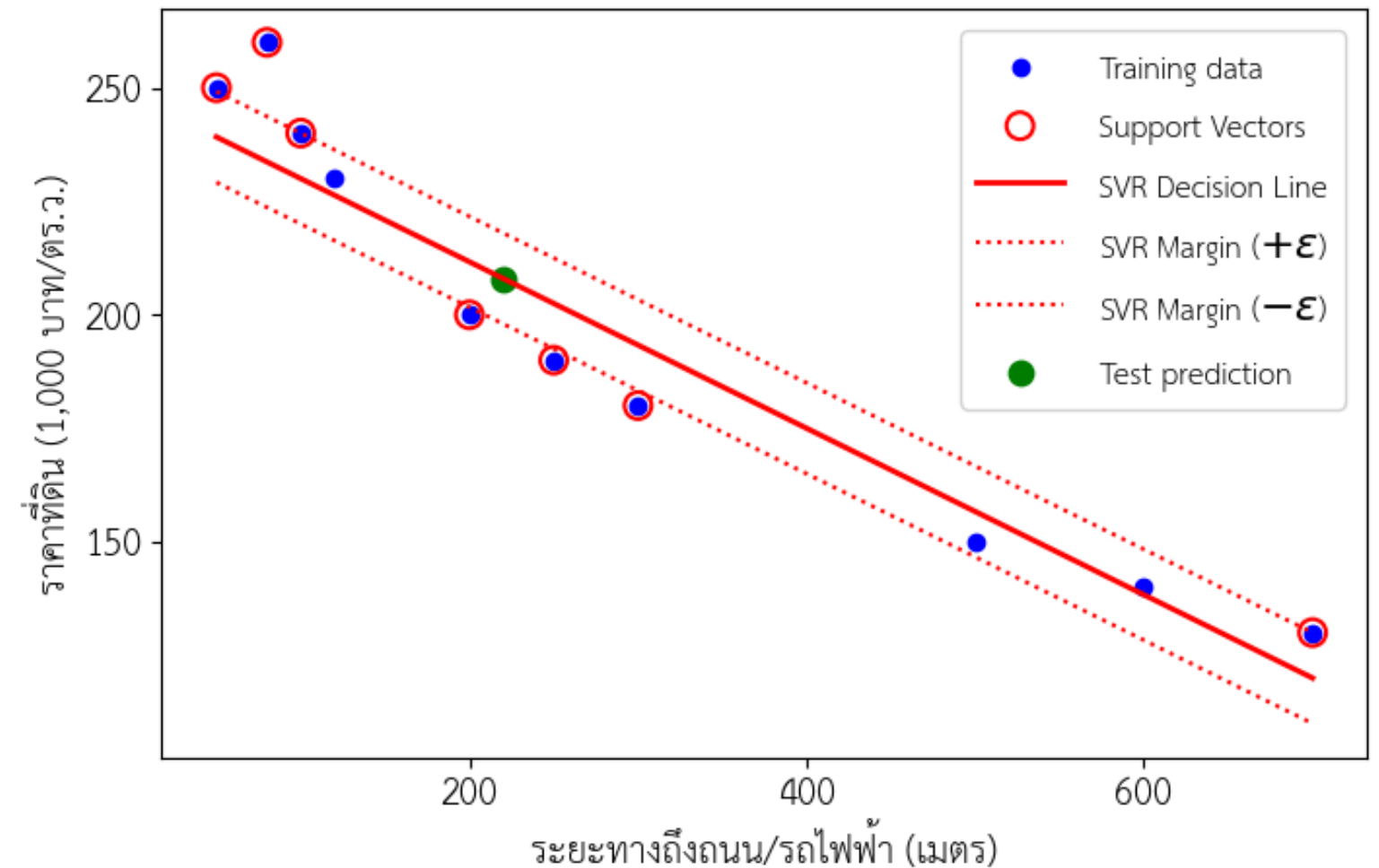
9.6.3 ตัวอย่างโค้ด SVR

```
1 #Import Python library
2 from sklearn.svm import SVR
3
4 #Load Data
5 x_train = np.array([50, 100, 80, 200, 500, 300, 700, 600, 250, 120]).reshape(-1, 1)
6 y_train = np.array([250, 240, 260, 200, 150, 180, 130, 140, 190, 230])
7 x_test = np.array([[220]])
8
9 #Build Model
10 net = SVR(kernel='linear', C=100, epsilon=10)
11 net.fit(x_train, y_train)
12
13 #Test Model
14 w = net.coef_[0]
15 b = net.intercept_[0]
16 y_pred = net.predict(x_test)
17
18 print("=== Test Result ===")
19 print(f"w = {w[0].round(4)}, b = {b:.2f}")
20 print(f"ราคาที่ดินที่ประเมินได้ คือ {y_pred[0]*1000:,.2f} บาท/ตร.ว.")
```

=== Test Result ===

w = -0.1833, b = 248.33

ราคาที่ดินที่ประเมินได้ คือ 208,000.00 บาท/ตร.ว.



9.7 แบบฝึกหัด

ตารางที่ 9-6 : ชุดข้อมูลราคาบ้าน (สำหรับข้อ 1-2)

ตารางที่ 9-6ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	7	0.4	3	139
2	4	0.2	7	159
3	3	0.6	4	60
4	10	1.2	2	25
5	8	1.4	1	26
6	3	0.5	5	65
7	2	0.2	3	100
8	3	0.1	7	160
9	2	0.3	2	134
10	9	0.9	1	40

ตารางที่ 9-6ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุบ้าน (ปี)	ระยะทางไปรถไฟฟ้า (กิโลเมตร)	จำนวนร้านสะดวกซื้อ (ร้าน)	ราคาบ้าน (1,000 บาท/ตร.ม.)
1	5	0.7	4	?

1. จงแสดงวิธีการคำนวณ KNN เพื่อประเมินราคาบ้าน **กำหนดให้** $k=3$

2. จงแสดงวิธีการคำนวณ MLP เพื่อประเมินราคาบ้าน

กำหนดให้

- Input Node = 3
- Hidden Node = 2 (Logistic Sigmoid)
- Output Node = 1 (Linear)
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = \begin{bmatrix} 3.003 & 2.012 \\ 0.485 & 0.283 \\ 1.944 & 1.955 \end{bmatrix}$
- Hidden Bias: $\mathbf{b}_h = [0.573, 1.072]$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = \begin{bmatrix} 65.472 \\ 65.005 \end{bmatrix}$
- Output Bias: $\mathbf{b}_y = [66.383]$

ตารางที่ 9-7 : ชุดข้อมูลราคาอสังหาริมทรัพย์ (สำหรับข้อ 3-4)

ตารางที่ 9-7ก : ชุดข้อมูลฝึก (Training Set)

รายการ	อายุ (ปี)	ราคาอสังหาริมทรัพย์ (1,000 บาท)
1	20	25
2	25	30
3	29	45
4	32	70
5	35	100
6	45	300
7	50	500
8	55	693
9	60	783
10	65	863
11	72	940
12	80	950

ตารางที่ 9-7ข : ชุดข้อมูลทดสอบ (Test Set)

รายการ	อายุ (ปี)	ราคาอสังหาริมทรัพย์ (1,000 บาท)
1	30	?
2	70	?

3. จงแสดงวิธีการคำนวณ OLS เพื่อประเมินราคาอสังหาริมทรัพย์

4. จงวาดกราฟความสัมพันธ์ระหว่างอายุและราคาอสังหาริมทรัพย์ แล้ววิเคราะห์ว่าควรใช้เทคนิคการเรียนรู้ของเครื่องไหนแทน OLS เพราะอะไร

ตารางที่ 9-8 : ชุดข้อมูลราคาหุ้น (สำหรับข้อ 5)

date	open	high	low	close
1/8/2568	100	102	98	99
4/8/2568	99	100	95	96
5/8/2568	96	98	95	97
6/8/2568	97	97	92	93
7/8/2568	93	95	91	92
8/8/2568	92	94	90	94
11/8/2568	94	94	88	89
12/8/2568	89	90	86	87
13/8/2568	87	88	85	88
14/8/2568	88	88	82	84

5. จงแสดงการคำนวณ RNN (Elman Network) เพื่อทำนายราคาหุ้น ณ วันที่ 15/8/2568

กำหนดให้

- Input Node = 4
- Hidden Node = 1 (Logistic Sigmoid)
- Output Node = 1 (ReLU)
- Time Step = 3
- Initial hidden state: $h_0 = 0$
- Hidden Weights (Input $\rightarrow$ Hidden): $\mathbf{W}_h = [0.01, -0.01, 0.02, -0.02]$
- Hidden Recurrent Weight: $\mathbf{U}_h = 0.5$
- Hidden Bias: $\mathbf{b}_h = -0.1$
- Output Weight (Hidden $\rightarrow$ Output): $\mathbf{W}_y = 100$
- Output Bias: $\mathbf{b}_y = 10$



THANK YOU FOR YOUR ATTENTION

 pichai_jo@rmutto.ac.th

